

A PRACTICAL TEXTBOOK & PORTFOLIO GUIDE

AI Engineering from Scratch to Shipped

Five production projects: Retrieval-Augmented Generation, Observability, Offline Small Language Models, Fine-Tuning (LoRA/QLoRA/DPO), and Real-Time Voice

Grounded in four working repositories with real, measured results.

Contents

Preface

- Who this book is for
- The three ways to read it
- A note on honesty of numbers
- The five projects and the four repositories

Chapter 1 — Foundations of Modern AI Engineering

- 1.1 What a large language model actually is
 - Tokens: the unit of everything
- 1.2 Embeddings and vector space
- 1.3 Transformers and attention (just enough)
- 1.4 Context windows and why chunking exists
- 1.5 Sampling and temperature
- 1.6 The three ways to make a model do your task
- 1.7 Evaluation: the discipline that makes it engineering
- 1.8 Real-time systems: a different world
- 1.9 How the rest of the book is organised
 - References

Chapter 2 — The Reproducible Environment

- 2.1 Reference hardware
- 2.2 Language, versions, and why we pin them
 - The one gotcha worth knowing up front
- 2.3 Creating and activating a virtual environment
- 2.4 Installing dependencies
- 2.5 Installing Ollama and pulling models (Project 2)
- 2.6 Docker (for deployment)
- 2.7 Environment variables and .env files
- 2.8 Validating the environment (health checks)
- 2.9 Starting, stopping, and cleaning up
- 2.10 Storage, context, and quantization settings
 - References

Chapter 3 — Git, GitHub, and Repository Security

- 3.1 Why version control, and why this way
- 3.2 Starting or cloning a repository
 - Commit identity
- 3.3 The everyday loop: branch → stage → commit → push
 - Writing meaningful commits
- 3.4 Keeping up to date: pull, rebase, merge, conflicts
- 3.5 Preventing secrets from ever being committed
 - Layer 1 — .gitignore
 - Layer 2 — .env.example (commit the shape, not the secret)
 - Layer 3 — Gitleaks (scan for secrets)
 - Layer 4 — Pre-commit hooks (block the leak before it happens)
- 3.6 Pull requests and code review

- 3.7 Continuous integration on every push
- 3.8 Tags, releases, and changelogs
- 3.9 Reverting, secret rotation, and repository settings
 - Branch protection (the settings that make CI mean something)
- 3.10 Dependency scanning and forks
 - References

Chapter 4 — Project 1: Production-Grade RAG

- 4.1 The problem RAG solves
- 4.2 Repository structure
- 4.3 Setup
- 4.4 Ingestion: from documents to a searchable index
 - Loading and normalising
 - Chunking: the most consequential knob
 - Embedding and storage
- 4.5 Retrieval: keywords, vectors, and fusion
 - Two complementary retrievers
 - Hybrid retrieval with Reciprocal Rank Fusion
 - Cross-encoder reranking
- 4.6 Generation with enforced citations
- 4.7 Prompt and context-window management
- 4.8 The golden dataset: methodology
- 4.9 Offline evaluation with Ragas
- 4.10 The other metrics, and how to compute them
- 4.11 Measured results
- 4.12 What worked, what didn't, and limitations
- 4.13 Run it end to end
 - References

Chapter 5 — Project 3: Monitoring and Observability

- 5.1 Why Projects 1 and 3 share a repository
- 5.2 The vocabulary: logs, metrics, traces, and friends
- 5.3 Instrumenting every stage
- 5.4 Choosing a tracing platform
- 5.5 Standing up Langfuse locally
- 5.6 The metrics rollup
- 5.7 Measured operational results
- 5.8 The dashboards to build
- 5.9 A disciplined anomaly workflow
- 5.10 The CI regression gate
 - References

Chapter 6 — Project 2: Offline Local SLM Lab

- 6.1 Why run a model locally at all
- 6.2 Setup and models
- 6.3 The application: CLI and API
- 6.4 Benchmark methodology
- 6.5 Measured benchmark results
- 6.6 Temperature and determinism study
- 6.7 Structured output that you can trust

6.8 Quantization and the quality-vs-speed tradeoff

6.9 Reproduce and record

References

Chapter 7 — Project 4: Fine-Tuning with LoRA, QLoRA, and DPO

7.1 The training vocabulary

7.2 Choosing a measurable task

7.3 The dataset pipeline

7.4 TRL vs. Axolotl

7.5 Configuration and hyperparameters

7.6 Training commands

7.7 Evaluation: base vs. SFT vs. SFT+DPO

What to expect and watch for (interview-ready)

7.8 Deciding QLoRA vs. full fine-tuning

7.9 One command on a GPU box

References

Chapter 8 — Project 5: Real-Time Multimodal (Voice)

8.1 Batch vs. streaming: a different world

8.2 The voice pipeline

8.3 Architecture and providers

8.4 Setup

8.5 The latency budget

8.6 Resilience: timeouts, fallbacks, degradation

8.7 Replay mode and failure injection

8.8 Run it

References

Chapter 9 — Testing Strategy

9.1 The testing pyramid, adapted for AI systems

9.2 The suites

Unit tests

Integration tests

End-to-end tests

Regression / evaluation tests (the AI-specific one)

Load tests

Failure-injection tests

Contract & schema-validation tests

Prompt tests

Retrieval tests

Security tests

9.3 Running everything

9.4 Reading a test summary

9.5 Troubleshooting the common failures

References

Chapter 10 — Deployment and Shipping

10.1 Containerising with Docker

10.2 Health checks and readiness

10.3 Docker Compose (multi-service)

10.4 Registries and the self-publishing pipeline

- 10.5 Deploying to the cloud (Azure Container Apps)
- 10.6 Secrets, configuration, logging, monitoring in production
- 10.7 Choosing a target: the tradeoff table
- 10.8 Scaling, cost control, rollback, releases
- References

Chapter 11 — Portfolio Readiness

- 11.1 The readiness checklist (apply to every project)
- 11.2 Turning “implemented” into “ready” — the concrete moves
- 11.3 The portfolio scorecard
 - Reading the scorecard honestly
- 11.4 The hub: tie it together

Chapter 12 — Hiring-Manager Presentation Guide

- 12.1 prod-rag (Projects 1 + 3)
- 12.2 local-slm-lab (Project 2)
- 12.3 llm-finetuning (Project 4)
- 12.4 realtime-voice (Project 5)
- 12.5 The question bank

Chapter 13 — Bibliography and Further Reading

- Foundations: transformers, tokens, embeddings
- Retrieval-Augmented Generation
- Evaluation and observability
- Local models and structured output
- Fine-tuning: LoRA, QLoRA, DPO
- Real-time systems
- Engineering practice: Git, CI, deployment, security
- Colophon

Preface

Who this book is for

This book is written for someone who is comfortable with basic programming and the command line, but who is **new to most of the ideas in modern AI engineering**: retrieval-augmented generation, embeddings, evaluation, tracing, fine-tuning, and real-time streaming systems. You do not need a machine-learning background. Every concept is introduced from first principles before any code appears.

It is also a **portfolio and interview guide**. The five projects it teaches are real, they live in public GitHub repositories, and — wherever a laptop can run the code — they carry *real, measured numbers* rather than illustrative ones. The final chapters show you how to present that work to a hiring manager.

The three ways to read it

1. **As a textbook.** Read front to back. Chapters 1–3 build the shared foundation (how language models, retrieval, and reproducible engineering work). Chapters 4–8 each teach one project. Chapters 9–13 cover the cross-cutting disciplines — testing, deployment, portfolio polish, and interview preparation.
2. **As an implementation guide.** Each project chapter is a complete, runnable walkthrough: prerequisites, setup, configuration, commands, expected output, tests, evaluation, benchmarks, and troubleshooting. Nothing is left as pseudocode where a laptop can run the real thing.
3. **As a reference.** The table of contents and the per-chapter *References* sections point at the authoritative documentation and papers behind every decision.

A note on honesty of numbers

A portfolio built on invented benchmarks is worse than no portfolio: it fails the first technical question. This book is scrupulous about labelling evidence. Throughout, you will see one of these tags:

MEASURED — a number produced by running the code on the hardware described in Chapter 2.

EXAMPLE — representative output shown to illustrate shape/format, not a benchmark. **REQUIRES**

YOU — a step that needs hardware, a paid API key, or a manual action only the repository owner can perform. The exact instructions are given, but the number is not yet collected.

Never present an **EXAMPLE** or a **REQUIRES YOU** target as a **MEASURED** result. That single discipline is what separates an engineer from a demo-builder.

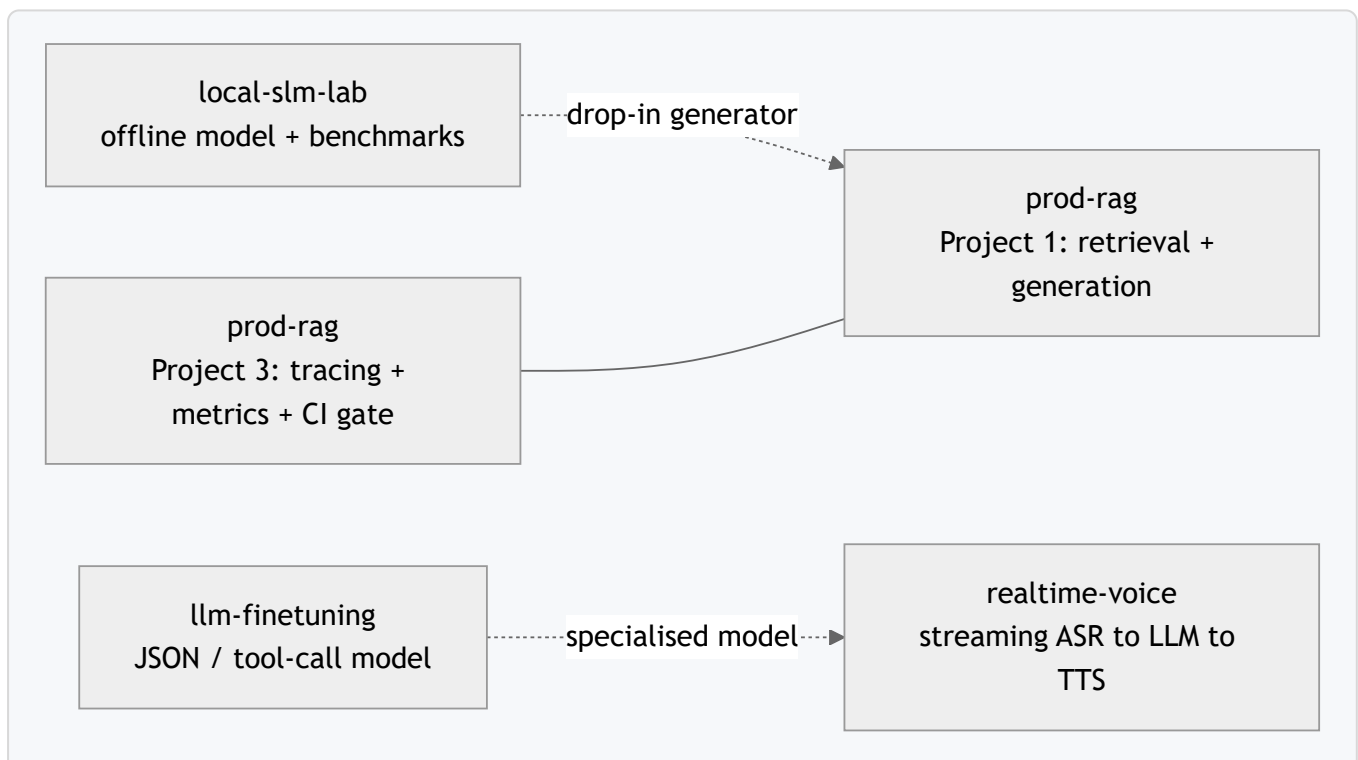
The five projects and the four repositories

The five projects map onto **four repositories**, because Projects 1 and 3 share one codebase by design (you cannot instrument a pipeline you have not built).

#	Project	Repository	Why
1	Production-grade RAG	prod-rag	The core retrieval + generation system.
3	Monitoring & observability	prod-rag	Observability <i>instruments Project 1's exact pipeline</i> — same code, added tracing/metrics/CI gates. Splitting them would mean maintaining a fake pipeline to observe.
2	Offline local SLM	local-slm-lab	Fully offline; no shared code with RAG, so it stands alone.
4	Fine-tuning (LoRA/QLoRA/DPO)	llm-finetuning	GPU training pipeline; independent lifecycle.
5	Real-time multimodal (voice)	realtime-voice	Streaming architecture; independent lifecycle.

The relationships between them are deliberate and worth stating in an interview:

- `local-slm-lab` produces a **benchmarked local model** that can serve as a drop-in generator inside `prod-rag` when privacy or cost rules out an API.
- `llm-finetuning` produces a **specialised JSON/tool-calling model** that is exactly the kind of component a `realtime-voice` assistant needs for fast, structured turns.
- `prod-rag` is where Projects 1 and 3 meet: retrieval quality (1) is only trustworthy because observability and CI gates (3) keep it from silently regressing.



Everything in this book — the code, the diagrams, the numbers, and this PDF's build pipeline — is reproducible from the repositories. Let us begin with the ideas the whole field rests on.

Chapter 1 — Foundations of Modern AI Engineering

Before building anything, we need a shared vocabulary. This chapter explains the handful of ideas that every later chapter assumes. Read it even if you have skimmed these words before, because the *engineering* meaning of each one — what it costs, where it breaks — is what matters here.

1.1 What a large language model actually is

A **large language model (LLM)** is a function that, given a sequence of text, predicts the probability distribution of the next chunk of text. That is the whole trick. Everything else — answering questions, writing code, holding a conversation — is that one operation applied repeatedly.

- **What it is:** a very large neural network (billions of parameters) trained on a very large text corpus to predict the next token.
- **Why it exists:** next-token prediction turns out to be a general-purpose objective. To predict the next word of a well-written explanation, the model must internalise grammar, facts, reasoning patterns, and style.
- **What problem it solves:** it replaces thousands of narrow, hand-built NLP systems (summarisers, classifiers, extractors) with one system you *program in natural language* via prompts.
- **How it works internally:** the network is a **transformer** (Section 1.3). It converts text into vectors, mixes information across positions with an *attention* mechanism, and emits a probability for every possible next token.
- **Where it fits:** the LLM is the “reasoning” box in every project in this book. In RAG it writes the grounded answer; in the voice assistant it turns a transcript into a reply; in fine-tuning it is the artefact we improve.

Tokens: the unit of everything

Models do not see characters or words; they see **tokens** — sub-word pieces produced by a *tokenizer*.

“Observability” might be three tokens; a rare word might be five. Two consequences drive real engineering decisions:

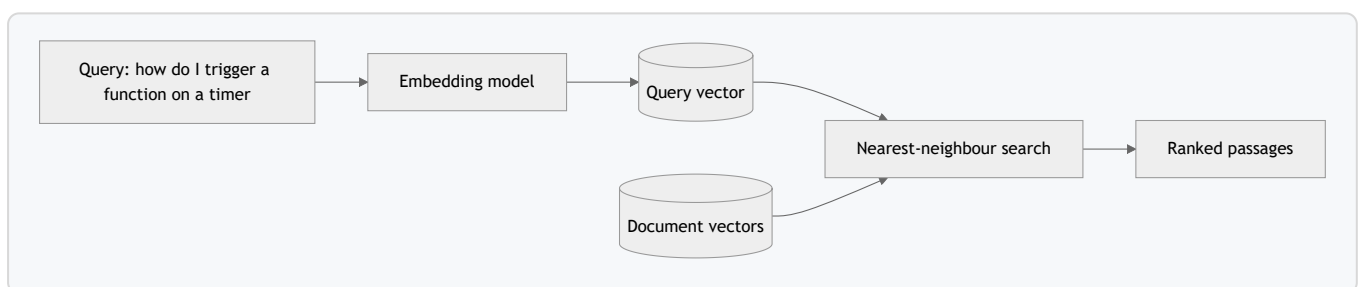
1. **You are billed and rate-limited per token**, split into *input* (prompt) and *output* (generation) tokens, usually at different prices. Cost control is token control.
2. **The context window is measured in tokens**, not words (Section 1.4).

Interview framing. “Why did you track input and output tokens separately?” Because they are priced differently and they tell different stories: input tokens measure how much context you are stuffing into the prompt (a retrieval problem), output tokens measure how verbose the model is (a prompt-design problem). In `prod-rag` we record both per request and turn them into a cost estimate — Chapter 5.

1.2 Embeddings and vector space

An **embedding** is a list of numbers (a vector, often 384–3072 dimensions) that represents the *meaning* of a piece of text, such that texts with similar meaning have vectors that are close together.

- **What it is:** a learned map from text to a point in high-dimensional space.
- **Why it exists:** computers cannot compare meaning directly, but they can compare vectors with simple arithmetic (cosine similarity).
- **What problem it solves:** “find me the passages most relevant to this question” becomes “find the stored vectors nearest to the question’s vector.”
- **How it works internally:** an embedding model (a cousin of the LLM) reads text and pools its internal representation into one vector. Training pushes paraphrases together and unrelated texts apart.
- **Where it fits:** embeddings are the heart of the *vector retrieval* half of RAG (Chapter 4). They are also how you can visualise a corpus (Section 1.2).
- **Industry use cases:** semantic search, deduplication, clustering, recommendation, retrieval for RAG.
- **Alternatives:** keyword search (BM25) compares *words*, not meaning; it wins on exact terms (error codes, product names) and loses on paraphrase. This is precisely why `prod-rag` uses **both** and fuses them (hybrid retrieval).



Mental model. Picture every passage as a star in a galaxy. The query is a new star; retrieval is “which existing stars are nearest?” Cosine similarity is the angle between two stars as seen from the origin — small angle, similar meaning.

1.3 Transformers and attention (just enough)

You can build every project here treating the transformer as a black box, but one idea — **attention** — explains the model’s strengths and its cost curve.

- **The problem before transformers:** older models (RNNs) read text strictly left to right, so information from far back faded.
- **Attention’s answer:** for each token, compute how much it should “attend to” every other token, and blend their representations accordingly. Every position can look at every other position in one step.
- **The cost:** attention is **quadratic** in sequence length — doubling the context roughly quadruples the compute for that layer. That is *why* context windows are finite and why stuffing more text into a prompt is not free (Section 1.4).

You do not need the maths to ship these projects, but the quadratic cost is the reason Chapter 4 spends effort on *chunking* and *context-window management* instead of “just send the whole document.”

1.4 Context windows and why chunking exists

The **context window** is the maximum number of tokens a model can consider at once (input + output). Modern models range from a few thousand to hundreds of thousands of tokens.

Two hard truths shape RAG:

1. **Corpora are bigger than any context window.** You cannot paste a 400-page manual into a prompt.
2. **More context is not always better.** Models exhibit a “lost in the middle” effect — relevant facts buried in a long context are used less reliably than the same facts placed near the top.

The engineering response is **chunking**: split documents into passages of a few hundred tokens, retrieve only the most relevant handful, and place them carefully in the prompt. Chapter 4 documents the exact chunk sizes tested in `prod-rag` (500–800 tokens, ~100-token overlap) and the results.

1.5 Sampling and temperature

When the model has a probability distribution over next tokens, **sampling** decides which token to actually emit. **Temperature** rescales that distribution:

- **Temperature 0** — always take the most likely token. Deterministic; reproducible; the right choice for extraction, classification, and tool calls.
- **Higher temperature (e.g. 0.7)** — flatten the distribution so less likely tokens get a chance. More varied and “creative”; the right choice for brainstorming; dangerous for structured output.

Chapter 6 (`local-slm-lab`) contains a **measured** temperature study: at temperature 0 the model produced **1 distinct output across 5 runs** for every prompt (fully deterministic); at 0.7 it produced **4–5 distinct outputs of 5**. That is not a vibe — it is a table you can point to.

1.6 The three ways to make a model do your task

There is a hierarchy of effort. Reach for the cheapest one that works.

Technique	What you change	Cost	When to use
Prompting	The input text only	Minutes	Almost always try first.
Retrieval (RAG)	Add <i>facts</i> to the prompt at runtime	Hours–days	The model lacks <i>knowledge</i> (your docs, fresh data).
Fine-tuning	The model’s <i>weights</i>	Days + GPU	The model lacks a <i>skill/format</i> even when given the facts.

- **RAG changes what the model *knows*** for one request without retraining. Use it for private or fast-changing knowledge.
- **Fine-tuning changes what the model *is***. Use it when you need a consistent behaviour or output format that prompting cannot reliably enforce — exactly the case Chapter 7 (`llm-finetuning`) targets with structured JSON extraction.

The classic interview trap. “Would you fine-tune to add company knowledge?” Usually **no** — knowledge that changes should live in a retrieval index (RAG), not baked into weights you would have to retrain. Fine-tune for *skill and format*, retrieve for *facts*.

1.7 Evaluation: the discipline that makes it engineering

Anyone can get one good answer from an LLM. The engineering question is: *how do you know it is good, and how do you keep it good as you change things?* That is **evaluation**, and it splits into two kinds you must not confuse:

- **Offline evaluation** — run the system over a fixed *golden dataset* of questions with known-good answers, and score it. Reproducible; used as a **CI gate** so a pull request that lowers quality fails the build (Chapters 4–5).
- **Online/operational monitoring** — watch *real* traffic in production: latency percentiles, cost, error rates, citation coverage (Chapter 5).

Key metrics you will meet:

- **Retrieval:** Precision@K, Recall@K, Mean Reciprocal Rank (MRR).
- **Generation:** faithfulness (is the answer supported by the retrieved text?), answer relevancy, context precision.
- **Operations:** p50/p90/p95/p99 latency, cost per request, failure rate.

Why percentiles, never averages. One 12-second request hidden behind a hundred fast ones vanishes in a mean but is exactly the user who churns. p90 (“90% of requests were at least this fast”) surfaces the tail. Chapter 5 shows the **measured** p50 = 2326 ms and p90 = 3709 ms from `prod-rag`.

1.8 Real-time systems: a different world

The first four projects are *request/response*: ask, wait, receive. Project 5 (`realtime-voice`) is **streaming**, and streaming breaks assumptions:

- Data arrives continuously (audio frames), not in one blob.
- Every stage has a **latency budget** measured in tens of milliseconds; the sum is the user’s perceived delay before they hear a reply.
- Dependencies *will* fail mid-stream, so you need **timeouts, fallbacks, and graceful degradation** rather than a single try/except.

Chapter 8 defines the streaming vocabulary (backpressure, load shedding, circuit breakers, replay). For now, hold onto the core idea: in a real-time system, **being late is the same as being wrong**.

1.9 How the rest of the book is organised

- **Chapter 2** — the reproducible environment: hardware, Python, Ollama, Docker, and the exact versions behind every measured number.
- **Chapter 3** — Git/GitHub workflow and repository security.
- **Chapter 4** — `prod-rag` , Project 1: retrieval + generation.
- **Chapter 5** — `prod-rag` , Project 3: observability + CI gating (same code).
- **Chapter 6** — `local-slm-lab` , Project 2: offline models + benchmarks.
- **Chapter 7** — `llm-finetuning` , Project 4: LoRA / QLoRA / DPO.
- **Chapter 8** — `realtime-voice` , Project 5: streaming ASR → LLM → TTS.
- **Chapters 9–13** — testing, deployment, portfolio readiness, interview preparation, and the bibliography.

References

- Vaswani et al., “Attention Is All You Need” (2017) — the transformer.
- Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks” (2020) — the RAG paper.
- Liu et al., “Lost in the Middle: How Language Models Use Long Contexts” (2023).
- Hugging Face, “LLM Course” and OpenAI, “Tokenizer” documentation.

Chapter 2 — The Reproducible Environment

A portfolio project that “works on my machine” is a liability. This chapter pins down exactly what “the machine” is, how to recreate the environment on Windows, Linux, and macOS, and how to validate that recreation with health checks. Every **MEASURED** number later in the book was produced on the hardware in Section 2.1 with the versions in Section 2.2.

2.1 Reference hardware

The benchmark numbers in Chapters 4–6 come from this laptop:

Component	Value
CPU	Intel Core Ultra 9 285H
System memory	63 GB
GPU	NVIDIA RTX 5070 Laptop GPU (+ Intel Arc 140T iGPU)
OS	Windows 11
Ollama	0.32.5
Python	3.14.4

REQUIRES YOU — GPU for fine-tuning. The four laptop-runnable projects were measured on the machine above. **Project 4 (`llm-finetuning`) needs a CUDA GPU with more VRAM than a laptop typically provides** (a 4B model in QLoRA wants ~16 GB). Chapter 7 gives a one-command script to run it on a rented GPU box; its before/after numbers are the one set this book marks **REQUIRES YOU**.

Rough guidance on what each project needs:

Project	CPU	GPU	RAM	Disk	Network
<code>prod-rag</code>	any modern	none (CPU embeddings)	8 GB+	~2 GB (models + index)	OpenAI key for generation/eval
<code>local-slm-lab</code>	any modern	helps, not required	16 GB+ (7B model)	~10 GB (3 models)	none (fully offline)
<code>llm-finetuning</code>	any	CUDA, ~16 GB VRAM	16 GB+	~30 GB (base + datasets + adapters)	Hugging Face (downloads)
<code>realtime-voice</code>	any modern	none	4 GB+	<1 GB	keys only for <i>live</i> mode

2.2 Language, versions, and why we pin them

Reproducibility means naming versions. This book uses **Python 3.14.4**. Each repository records its expected interpreter in a `.python-version` file and its dependency ranges in `pyproject.toml`.

Why pin at all? A silent minor-version bump in a dependency can change a benchmark or break an import. Pinning turns “it broke and I don’t know why” into “the diff shows exactly what changed.” Chapter 3 shows how CI enforces the pinned versions on every push.

The one gotcha worth knowing up front

`local-slm-lab`, `prod-rag`, and `realtime-voice` install cleanly on Python 3.14. `llm-finetuning` depends on the deep-learning stack (PyTorch, TRL, PEFT), whose wheels track the CUDA toolchain — install PyTorch **from the CUDA index** that matches your GPU driver (Chapter 7), not from PyPI’s default CPU wheel.

2.3 Creating and activating a virtual environment

A **virtual environment** (`venv`) isolates one project’s packages from the rest of your system, so two projects can depend on different versions without conflict.

Windows (PowerShell):

```
cd prod-rag
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python --version          # expect: Python 3.14.4
```

Linux / macOS (bash/zsh):

```
cd prod-rag
python3 -m venv .venv
source .venv/bin/activate
python --version          # expect: Python 3.14.4
```

Deactivate any time with `deactivate`. Repeat per repository — each has its own `.venv` (which is git-ignored; never commit it).

2.4 Installing dependencies

Every repository is an installable package with a `dev` extra (tests + linters):

```
pip install -e ".[dev]"
```

-e installs in *editable* mode: your source edits take effect without reinstalling. The optional `[dev]` group adds `pytest` and `ruff`. `prod-rag` also defines a `[serve]` extra (FastAPI + Uvicorn) for the container, and a `[cohere]` extra for the optional hosted reranker.

EXAMPLE — successful install (tail).

```
Successfully built prod-rag
Installing collected packages: prod-rag
Successfully installed prod-rag-0.1.0
```

Each repo also ships a `Makefile` so the common tasks are one word:

```
make setup      # pip install -e ".[dev]"
make lint       # ruff check .
make test       # pytest
```

2.5 Installing Ollama and pulling models (Project 2)

Ollama runs open-weight models locally with a single binary and an OpenAI-compatible HTTP endpoint on `localhost:11434`.

- **Windows/macOS:** download the installer from <https://ollama.com/download>.
- **Linux:** `curl -fsSL https://ollama.com/install.sh | sh`.

Verify and pull the three models the benchmark uses:

```
ollama --version      # expect: ollama version is 0.32.5
ollama pull llama3.2   # 3B
ollama pull phi4-mini  # ~4B
ollama pull mistral    # 7B
ollama list            # confirm all three are present
```

EXAMPLE — `ollama list`.

NAME	ID	SIZE	MODIFIED
llama3.2:latest	a80c4f17acd5	2.0 GB	...
phi4-mini:latest	1a2b3c...	2.5 GB	...
mistral:latest	f974a74358d6	4.1 GB	...

A quick smoke test that the server answers:

```
ollama run llama3.2 "Reply with the single word: ok"
```

2.6 Docker (for deployment)

Docker packages an app plus its dependencies into a portable image. You need it only for Chapter 10 (deployment) and the `prod-rag` container. Verify:

```
docker --version          # e.g. Docker version 27.x
docker compose version    # for the Langfuse stack in Chapter 5
```

`prod-rag` ships a `Dockerfile` (a FastAPI server) and a `docker-compose.langfuse.yml` (the self-hosted tracing stack). Both are covered where they are used.

2.7 Environment variables and `.env` files

Secrets (API keys) must **never** be hard-coded or committed. Each repo reads them from environment variables, loaded from a git-ignored `.env` file via `python-dotenv`. Every repo ships a committed `.env.example` template:

```
cp .env.example .env          # then edit .env with your real keys
```

REQUIRES YOU — API keys. Where to obtain each:

- **OpenAI** (`OPENAI_API_KEY`) — `platform.openai.com` → API keys. Used by `prod-rag` for generation and by the Ragas eval judge. Paid, usage-based.
- **Cohere** (`COHERE_API_KEY` , optional) — `dashboard.cohere.com`. Hosted reranker; `prod-rag` falls back to a local cross-encoder if absent.
- **Langfuse** (`LANGFUSE_*`) — self-hosted (Chapter 5) or `cloud.langfuse.com`.
- **Deepgram / ElevenLabs / OpenAI or Anthropic** — for *live* `realtime-voice` (Chapter 8). The offline demo needs none of them.
- **Hugging Face** (`HF_TOKEN`) — `huggingface.co/settings/tokens`, for `llm-finetuning` dataset/model downloads and pushing adapters.

Section 3.5 covers keeping these out of git for good (gitleaks + pre-commit).

2.8 Validating the environment (health checks)

Before running anything substantial, confirm the pieces are wired up.


```
# Python & package import
python -c "import sys; print(sys.version)"
python -c "import rag; print('prod-rag import OK')"      # from prod-rag, PYTHONPATH=src

# Ollama server reachable
curl http://localhost:11434/api/tags                    # JSON list of models

# Lint + tests green
make lint
make test
```

EXAMPLE — make test summary.

```
===== test session starts =====
collected 9 items
tests/test_citations.py .... [ 44%]
tests/test_chunker.py ... [ 77%]
tests/test_metrics.py .. [100%]
===== 9 passed in 1.42s =====
```

2.9 Starting, stopping, and cleaning up

```
# prod-rag API locally
make serve          # uvicorn api:app --reload (http://localhost:8000)

# Langfuse tracing stack (Chapter 5)
docker compose -f docker-compose.langfuse.yml up -d
docker compose -f docker-compose.langfuse.yml down      # stop + remove containers
docker compose -f docker-compose.langfuse.yml down -v   # ALSO delete volumes (wipes traces)

# free disk: remove a local model you no longer need
ollama rm mistral
```

2.10 Storage, context, and quantization settings

- **Storage budget:** three Ollama models $\approx$ 10 GB; `prod-rag`’s Chroma index + embeddings $\approx$ 1–2 GB; `llm-finetuning` base model + datasets + adapters $\approx$ 30 GB.
- **Context window:** `prod-rag` chunks at 500–800 tokens and retrieves a small handful, so prompts stay well under any modern model’s window (Chapter 4).
- **Quantization:** Ollama serves models quantized by default (e.g. `Q4_K_M`), which is why a 7B model fits in $\sim$ 5 GB of memory (Chapter 6’s table). QLoRA (Chapter 7) uses 4-bit quantization of the base model during training.

With the environment reproducible and validated, we can adopt the professional Git habits that make the whole thing credible — Chapter 3.

References

- Ollama documentation (ollama.com/docs), Python `venv` docs, `python-dotenv` , Docker & Docker Compose documentation, PyTorch “Get Started” (CUDA wheels).

Chapter 3 — Git, GitHub, and Repository Security

Hiring managers read your commit history before they read your code. A clean history, protected main branch, green CI, and *zero* leaked secrets say “professional” before a single function is reviewed. This chapter is the workflow used across all four repositories.

3.1 Why version control, and why this way

- **What Git is:** a distributed version-control system that records snapshots (commits) of your project and lets many lines of work (branches) coexist.
- **Why it exists:** to make change *safe* — every state is recoverable, every change is attributable, and collaboration does not overwrite work.
- **What problem it solves here:** it is the substrate for CI gates (Chapter 5), release management (Chapter 10), and the audit trail a reviewer trusts.

3.2 Starting or cloning a repository

```
# Start fresh
git init
git branch -M main
git remote add origin https://github.com/mimuruth/prod-rag.git

# Or clone an existing one
git clone https://github.com/mimuruth/prod-rag.git
cd prod-rag
```

Commit identity

These repositories set the author **inline per commit** rather than relying on global config, which guarantees the right identity even on a shared machine:

```
git -c user.name="mimuruth" -c user.email="mimuruth@users.noreply.github.com" \
  commit -m "feat: add hybrid retrieval"
```

Using the GitHub `noreply` email keeps your real address out of the public log.

3.3 The everyday loop: branch → stage → commit → push

Never commit straight to `main`. Branch per unit of work:

```
git checkout -b feat/reranker      # create + switch to a feature branch
# ... edit files ...
git status                        # what changed?
git diff                          # review the exact changes
git add src/rag/retrieve/rerank.py # stage specific files (prefer over `git add -A`)
git commit -m "feat: add cross-encoder reranker with local fallback"
git push -u origin feat/reranker  # publish the branch
```

Writing meaningful commits

Follow **Conventional Commits**: `type(scope): summary`. Types you will use: `feat`, `fix`, `docs`, `test`, `refactor`, `ci`, `chore`. A good message says *what changed and why*, not “update file.”

EXAMPLE — a readable history.

```
feat: result badges, results chart, ACA deploy scaffold, Makefile
ci: build + push container image to GHCR on version tag
fix: fold token usage into span metadata for Langfuse v4
docs: add architecture diagram
```

3.4 Keeping up to date: pull, rebase, merge, conflicts

```
git checkout main
git pull origin main          # fetch + merge others' work

git checkout feat/reranker
git rebase main               # replay your commits on top of latest main
```

Rebase vs. merge. Rebase produces a linear, readable history (preferred for feature branches before opening a PR). Merge preserves the exact branch topology (used when integrating a reviewed PR). If a rebase hits a **conflict**:

```
# Git marks conflicts in the file with <<<<<< ===== >>>>>>
# Edit to the desired result, then:
git add path/to/conflicted_file.py
git rebase --continue          # or: git rebase --abort to back out
```

3.5 Preventing secrets from ever being committed

This is the single most important security habit. Four layers, weakest to strongest:

Layer 1 — `.gitignore`

Every repo ignores secrets, environments, caches, and build output:

```
.env
.venv/
__pycache__/
.metrics/
.chroma/
build/
*.log
```

Layer 2 — `.env.example` (commit the shape, not the secret)

Commit a template with empty values so collaborators know what keys exist:

```
# .env.example
OPENAI_API_KEY=
COHERE_API_KEY=
LANGFUSE_PUBLIC_KEY=
LANGFUSE_SECRET_KEY=
LANGFUSE_HOST=http://localhost:3000
```

Layer 3 — Gitleaks (scan for secrets)

Gitleaks scans your history and working tree for anything that looks like a key. Install and run:

```
# install (macOS: brew install gitleaks; or download a release binary)
gitleaks detect --source . --verbose
```

EXAMPLE — a clean scan.

```
INF scanned ~180 commits
INF no leaks found
```

EXAMPLE — a caught leak (what failure looks like).

```
WRN leaks found: 1
Finding: OPENAI_API_KEY=sk-*****
File:     notebooks/scratch.ipynb   Rule: openai-api-key
```

When this fires, you *rotate the key* (Section 3.9) — removing it from the file is not enough, because it is already in history and possibly already scraped.

Layer 4 — Pre-commit hooks (block the leak before it happens)

A **pre-commit hook** runs checks automatically before each commit. Every repo pins `gitleaks` (v8.18.4) in

```
.pre-commit-config.yaml :
```

```
repos:
- repo: https://github.com/gitleaks/gitleaks
  rev: v8.18.4
  hooks:
  - id: gitleaks
```

```
pip install pre-commit
pre-commit install          # wire the hook into .git/hooks
pre-commit run --all-files  # run on demand
```

Now a commit that contains a secret is rejected locally — the strongest possible place to stop it.

3.6 Pull requests and code review

```
# push the branch, then open a PR (GitHub CLI)
gh pr create --fill --base main --head feat/reranker
```

A PR is where CI runs (lint, tests, and the eval gate — Chapter 5), where a reviewer comments, and where the *squash-merge* happens so `main` gets one clean commit per feature:

```
gh pr merge --squash --delete-branch
```

3.7 Continuous integration on every push

Each repo has GitHub Actions workflows under `.github/workflows/`. `prod-rag` runs four: `lint-test.yml`, `eval.yml` (the quality gate), `gitleaks.yml`, and `docker.yml` (image build on tag). A minimal lint-test job:

```
name: lint-test
on: [push, pull_request]
jobs:
  lint-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.11" }
      - run: pip install pytest ruff
      - run: ruff check .
      - run: pytest -q
```

EXAMPLE — a green run vs. a failed run.

```
✓ lint-test (ubuntu-latest) 32s
✓ gitleaks (ubuntu-latest) 11s
X rag-eval-gate 58s faithfulness 0.71 < threshold 0.80
```

The third line is Chapter 5's regression gate doing its job: the PR is blocked until quality recovers.

3.8 Tags, releases, and changelogs

Tag meaningful milestones with **semantic versioning** (`vMAJOR.MINOR.PATCH`):

```
git tag -a v1.0.0 -m "v1.0.0 - production RAG + observability + CI gating"
git push origin v1.0.0
gh release create v1.0.0 --generate-notes
```

In `prod-rag`, pushing a `v*` tag also triggers `docker.yml`, which builds and publishes the container image to GHCR (Chapter 10). Keep a `CHANGELOG.md` grouped by release with `Added` / `Changed` / `Fixed` sections (the “Keep a Changelog” format), or let `--generate-notes` draft it from PR titles.

3.9 Reverting, secret rotation, and repository settings

- **Undo a bad commit safely (already pushed):** `git revert <sha>` creates a new commit that inverts the change — history stays intact (preferred over `reset --hard` on shared branches).
- **Secret rotation:** if a key is ever exposed, *revoke it at the provider and issue a new one*. Then update your local `.env` and the GitHub Actions secret. Scrubbing the file is necessary but **not sufficient** — assume any committed key is compromised.
- **GitHub Actions secrets:** store CI credentials as encrypted repo secrets, not in code: `gh secret set OPENAI_API_KEY --repo mimuruth/prod-rag`. The workflow reads them as `${{ secrets.OPENAI_API_KEY }}`.

Branch protection (the settings that make CI mean something)

On GitHub → Settings → Branches → protect `main`:

- Require a pull request before merging.
- Require status checks to pass (`lint-test`, `gitleaks`, `rag-eval-gate`).
- Require branches to be up to date before merging.
- Optionally require a review and signed commits.

Without protection, a green CI is advisory; with it, a red check *blocks* the merge — which is the entire point of Chapter 5’s eval gate.

3.10 Dependency scanning and forks

- **Dependency scanning:** enable GitHub Dependabot alerts, and for Python run `pip-audit` in CI to flag known-vulnerable packages.
- **Forks:** when contributing to a repo you cannot push to, fork it, add the original as an `upstream` remote, branch, push to your fork, and open a PR across repositories:

```
git remote add upstream https://github.com/ORG/repo.git
git fetch upstream && git rebase upstream/main
```

With reproducible environments (Chapter 2) and professional Git hygiene (Chapter 3) in place, we can build the first system: production-grade RAG.

References

- Pro Git (Chacon & Straub); Conventional Commits; Keep a Changelog; Semantic Versioning (semver.org); Gitleaks and pre-commit documentation; GitHub Actions, branch protection, and Dependabot documentation.

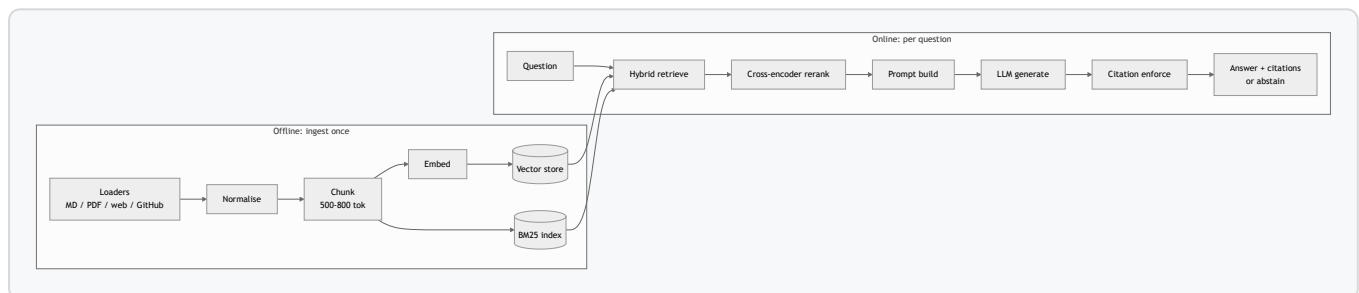
Chapter 4 — Project 1: Production-Grade RAG

Repository: `prod-rag` · **What you will build:** a domain-specific “Ask My Docs” system that retrieves from a real corpus, reranks, generates an answer *with citations*, and **refuses to answer** when the evidence is insufficient — then proves its quality with an offline evaluation suite that runs as a CI gate.

4.1 The problem RAG solves

An LLM’s knowledge is frozen at training time and it has never seen your private documents. Ask it about *your* API and it will either say it does not know or, worse, *confidently invent* an answer (a **hallucination**).

Retrieval-Augmented Generation fixes this by changing the *input*, not the model: at question time, fetch the most relevant passages from your corpus and place them in the prompt with an instruction to answer *only* from them. The model becomes a reader-and-synthesiser over evidence you control.



4.2 Repository structure

```

prod-rag/
  src/rag/
    ingest/      # loaders + chunker
    index/       # vector_store.py (Chroma) + bm25.py
    retrieve/    # hybrid.py (RRF fusion) + rerank.py (cross-encoder)
    generate/    # answer.py (pipeline) + citations.py (enforcement)
    observability/ # tracing.py + metrics.py (Chapter 5)
  prompts/      # answer.yaml – versioned prompt (treated as code)
  config/       # retrieval.yaml – params + eval thresholds
  eval/         # run_ragas.py + golden/qa_pairs.jsonl
  docs/         # the corpus (Azure Learn docs + a system doc)
  api.py        # FastAPI wrapper (Chapter 10)
  .github/workflows/eval.yml # the CI quality gate (Chapter 5)
  
```

4.3 Setup

```

git clone https://github.com/mimiruth/prod-rag.git && cd prod-rag
python -m venv .venv && .\.venv\Scripts\Activate.ps1 # Windows
pip install -e ".[dev]"
cp .env.example .env # add OPENAI_API_KEY
  
```

REQUIRES YOU — `OPENAI_API_KEY` . Generation and the evaluation judge call `gpt-4o-mini` . Retrieval itself (embeddings + BM25 + the local reranker) runs without any key, so you can develop and test retrieval offline; only the final answer synthesis and Ragas scoring need the key.

4.4 Ingestion: from documents to a searchable index

Loading and normalising

The loaders read Markdown, PDF (`pypdf`), web pages, and GitHub sources into a common record shape `{text, metadata}` . Normalisation strips boilerplate and attaches metadata (`source` , `title`) that later becomes the **citation**.

The shipped corpus in `docs/` is four real **Azure Learn** documents (Azure Functions, Container Apps, Blob Storage) plus a document describing the RAG system itself — a small but genuine corpus, which is why the numbers in Section 4.11 are honest rather than inflated by a toy dataset.

Chunking: the most consequential knob

- **What it is:** splitting each document into passages of a few hundred tokens.
- **Why it exists:** documents exceed the context window, and retrieval is more precise over small passages than whole files (Section 1.4).
- **How chunk size trades off:**
 - *Too small* (e.g. 128 tokens) — a passage loses the surrounding context that makes it answerable; recall of complete facts drops.
 - *Too large* (e.g. 2000 tokens) — each hit drags in irrelevant text, diluting the prompt and raising cost; precision drops.
 - **Overlap** (~100 tokens) copies a little text between neighbours so a fact straddling a boundary is not cut in half.

`prod-rag` uses **500–800 token chunks with ~100-token overlap**, configured in `config/retrieval.yaml` . This range is the widely used sweet spot for prose documentation: large enough to be self-contained, small enough to stay precise.

Interview framing — “How did you choose chunk size?” “I treated it as a retrieval hyperparameter. Smaller chunks raise precision but fragment facts; larger chunks raise recall but dilute the prompt and cost more. For prose docs the 500–800 token band with ~100 overlap is the empirical sweet spot, and I made it a config value so it is tunable against the eval suite rather than hard-coded.”

Embedding and storage

Each chunk is embedded and stored in **ChromaDB** (a local vector store). Build the indexes:

```
python -m rag.ingest.loaders --source docs/
```

EXAMPLE — ingest output.

```
Loaded 5 documents -> 63 chunks (500-800 tok, 100 overlap)
Embedded 63 chunks -> Chroma collection 'docs' (persisted to .chroma/)
BM25 index built over 63 chunks -> .bm25/
```

4.5 Retrieval: keywords, vectors, and fusion

Two complementary retrievers

- **BM25 (lexical):** ranks by *word overlap* with clever term weighting. Strong on exact tokens — error codes, API names, rare terms. Blind to paraphrase.
- **Vector (semantic):** ranks by *embedding similarity*. Strong on paraphrase (“timer trigger” ≈ “run on a schedule”). Can miss an exact rare token.

Neither dominates. A question with a specific product name *and* a paraphrased intent needs both.

Hybrid retrieval with Reciprocal Rank Fusion

`prod-rag` runs both retrievers and fuses their ranked lists with **Reciprocal Rank Fusion (RRF)**, implemented in `retrieve/hybrid.py`. RRF scores each document by summing $1 / (k + \text{rank})$ across the lists in which it appears:

$$\text{RRF}(d) = \sum_i \frac{1}{k + \text{rank}_i(d)}$$

RRF is rank-based, so it needs no score calibration between the two very different scales (BM25’s term score vs. cosine similarity) — a document ranked highly by *either* retriever floats up; ranked highly by *both*, higher still.

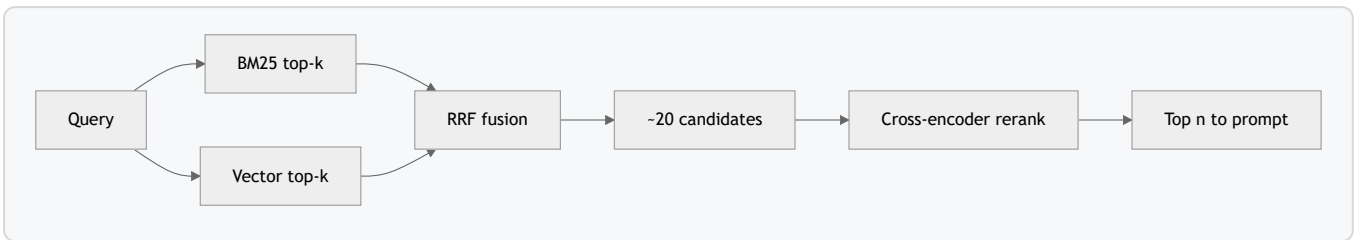
Alternatives considered. Weighted score blending (needs per-corpus tuning of the weight and score normalisation) and a single retriever (simpler, but gives up one of the two strengths). RRF was chosen because it is parameter-light (k only), robust, and the standard baseline for hybrid fusion.

Cross-encoder reranking

Retrieval returns, say, the top 20 fused candidates. A **cross-encoder reranker** then reads each (*query*, *passage*) pair *together* and scores true relevance, reordering to the top n .

- **Bi-encoder (retrieval):** embeds query and passage *separately* — fast, scales to millions, but never sees them together.
- **Cross-encoder (rerank):** processes the pair jointly — far more accurate, too slow for the whole corpus, perfect for re-ranking a shortlist.

`retrieve/rerank.py` uses a hosted **Cohere Rerank** when `COHERE_API_KEY` is set and *falls back to a local sentence-transformers cross-encoder* otherwise — wrapped in a `try/except ImportError` so the system degrades gracefully instead of crashing when the optional dependency is absent.



4.6 Generation with enforced citations

`generate/answer.py` orchestrates the online path in one function, `answer(question) -> {answer, citations, refused}` :

1. **Retrieve** (hybrid) → **rerank** → top- n chunks.
2. **Build the prompt** from the versioned template in `prompts/answer.yaml` , inserting each chunk with its citation id and source.
3. **Generate** with `gpt-4o-mini` under a grounding instruction.
4. **Enforce citations** (`citations.py`): verify the answer's cited ids correspond to retrieved chunks. If the answer is not grounded, **abstain** with a fixed refusal rather than risk a hallucination.

Why abstention is a feature, not a bug. In a production doc-assistant, a confident wrong answer is more expensive than “I don’t have enough information to answer that,” because it erodes trust and can mislead action. Enforced grounding + abstention is the mechanism that turns “usually right” into “trustworthy.”

Ask a question:

```
python -m rag.generate.answer "How do I trigger an Azure Function on a timer?"
```

EXAMPLE — a grounded answer.

```
You configure a timer trigger with a CRON expression in the function's
binding. [c12]
Citations:
  [c12] docs/azure-functions.md - "Timer trigger" section
refused: false
```

EXAMPLE — an abstention (out-of-corpus question).

```
I don't have enough information to answer that.
refused: true
```

4.7 Prompt and context-window management

The prompt is **versioned** in `prompts/answer.yaml` and treated as code: a change to wording is a diff that CI evaluates, so you never silently regress the system by “just tweaking the prompt.” Only the reranked top- `n` chunks enter the context, keeping prompts small (well under the model’s window), cheap, and free of the “lost in the middle” effect from Section 1.4.

4.8 The golden dataset: methodology

Evaluation is only as trustworthy as its dataset. `eval/golden/qa_pairs.jsonl` holds **18 golden QA pairs** (6 about the RAG system itself, 12 about the Azure docs). The methodology:

- **Question selection:** cover the corpus’s real information needs — factual lookups, “how do I...” tasks, and a few that are *deliberately unanswerable* from the corpus, to test abstention.
- **Expected answers:** written from the source text, each tied to the specific passage that supports it (the evidence).
- **Leakage prevention:** the golden answers are derived from the *documents*, and the system is scored on retrieving+using those documents — the model is never shown the golden answer, so there is no train/test leakage. When you grow the set, keep any tuning decisions (thresholds, chunk size) made on a *validation* slice separate from the held-out *test* slice used for the final number.

4.9 Offline evaluation with Ragas

`eval/run_ragas.py` ingests the corpus, runs the full pipeline over the 18 questions, and scores three metrics with **Ragas** (an LLM-as-judge framework), using `gpt-4o-mini` as the judge and `text-embedding-3-small` for the embedding metrics:

- **Faithfulness** — is every claim in the answer supported by the retrieved context? (Directly measures hallucination.)
- **Answer relevancy** — does the answer actually address the question?
- **Context precision** — are the retrieved chunks relevant (not padding)?

```
python eval/run_ragas.py
```

4.10 The other metrics, and how to compute them

The shipped harness reports the three Ragas metrics above. The broader RAG metric family — and how you would compute each — is worth knowing for interviews:

Metric	Question it answers	How to compute
Precision@K	Of the K retrieved, how many are relevant?	Label relevance per golden question; $\text{relevant} / \text{retrieved}$.
Recall@K	Of all relevant, how many did we retrieve?	$\text{relevant} / \text{retrieved}$.
MRR	How high is the first relevant hit?	Mean of $1 / \text{rank_of_first_relevant}$.
Faithfulness	Is the answer grounded?	Ragas (measured , 4.11).
Answer relevancy	Does it address the question?	Ragas (measured).
Context precision	Are retrieved chunks on-topic?	Ragas (measured).
Citation coverage	Fraction of answers carrying valid citations	Enforcement layer (measured , Chapter 5).
Cost per query	\$ per answer	Token counts $\times$ price (measured , Chapter 5).

Honesty note. Precision@K / Recall@K / MRR require per-question relevance labels the current 18-pair set does not yet carry, so this book does **not** print numbers for them — it shows the method. The Ragas three and the operational metrics *are* measured and shown next.

4.11 Measured results

MEASURED — Ragas over the 18-pair golden set (gpt-4o-mini judge).

Metric	Score	CI gate threshold
Faithfulness	0.87	≥ 0.80
Answer relevancy	0.85	≥ 0.78
Context precision	1.00	≥ 0.80

Reading the numbers:

- **Context precision 1.00** — over this corpus, the retrieved chunks were consistently on-topic; hybrid + rerank is doing its job of not dragging in noise.
- **Faithfulness 0.87** — high grounding; the citation-enforcement design shows up here. The gap from 1.0 is where an answer generalised slightly beyond the retrieved text — exactly the kind of regression the gate watches.
- **Answer relevancy 0.85** — answers address the questions well; this rises as the golden set grows and the prompt is tuned against it.

Thresholds are set **just below** the measured baseline (0.80 / 0.78 / 0.80) so that ordinary noise passes but a real regression fails the build (Chapter 5).

4.12 What worked, what didn't, and limitations

- **Hybrid beat either retriever alone** on questions mixing an exact term with a paraphrased intent — the motivating case for fusion.
- **Reranking changed the top order**, promoting the passage that actually answered the question above lexically-similar-but-off-topic neighbours.
- **Citation enforcement reduced hallucination** by construction: an answer that cannot cite retrieved evidence is refused, which is why faithfulness is high.
- **Limitations:** the corpus and golden set are small (18 pairs), so the numbers are a *baseline*, not a leaderboard claim; Precision@K-style metrics await relevance labels; a single embedding model and reranker are used (no ensemble).

4.13 Run it end to end

```
make setup
python -m rag.ingest.loaders --source docs/      # build indexes
python -m rag.generate.answer "What is Azure Container Apps?"
python eval/run_ragas.py                        # score the golden set
make test                                       # unit tests
```

With a working, evaluated RAG system, Chapter 5 makes it *observable* and wires the eval suite into CI as a regression gate — Project 3.

References

- Lewis et al. (2020), RAG; Robertson & Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond”; Cormack et al. (2009), Reciprocal Rank Fusion; Nogueira & Cho, “Passage Re-ranking with BERT”; Ragas documentation; ChromaDB and sentence-transformers documentation.

Chapter 5 — Project 3: Monitoring and Observability

Repository: `prod-rag` (same codebase as Chapter 4) · **What you will build:** end-to-end tracing, a metrics rollout with real latency/cost/quality numbers, and a CI gate that fails a pull request when quality regresses.

5.1 Why Projects 1 and 3 share a repository

You cannot instrument a pipeline you have not built. Observability is the RAG pipeline with tracing spans and metric records threaded through it. Splitting them into two repositories would force you to maintain a fake pipeline to observe. Instead, the git history tells the story: `v0.2` is Project 1 (the pipeline), `v1.0` is Project 3 (the same pipeline, now observable and gated).

5.2 The vocabulary: logs, metrics, traces, and friends

These words are used loosely in the wild; precision matters in an interview.

Term	What it is	Answers
Log	A timestamped text/JSON line about one event	“What happened here?”
Metric	A number aggregated over time (counter, gauge, histogram)	“How much / how often / how fast overall?”
Event	A discrete, structured thing that occurred	“What just happened, with what fields?”
Span	A single timed operation with start/end + attributes	“How long did <i>this stage</i> take?”
Trace	A tree of spans for one request	“Where did the whole request spend its time?”
Correlation ID	A shared id linking all records of one request	“Show me everything for request X.”
Distributed tracing	Traces that span multiple services	“Which service in the chain was slow?”

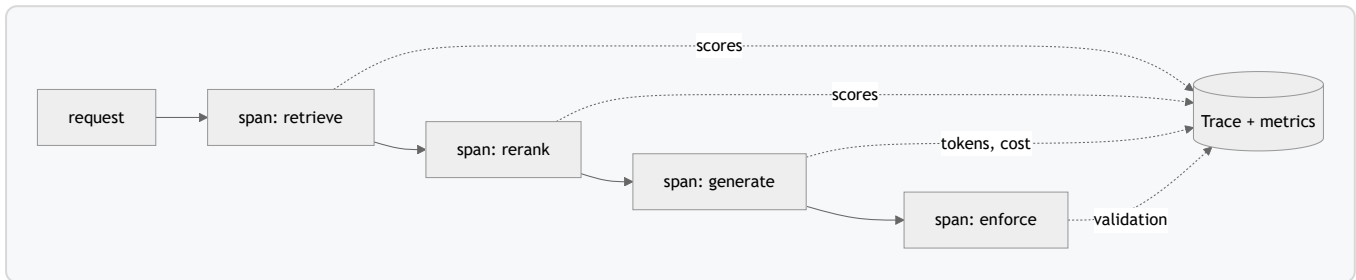
- **Offline evaluation** (Chapter 4) scores a fixed golden set — reproducible, used as a gate.
- **Online/operational monitoring** watches live traffic — latency, cost, errors.
- **Quality monitoring** watches *answer quality* on live traffic (citation coverage, faithfulness sampling), not just system health.

5.3 Instrumenting every stage

`observability/tracing.py` wraps each pipeline stage in a span; the whole `answer()` call is one trace. For each request, the system captures:

- **Query** and any query transformation.

- **Retrieved chunks**, their **retrieval scores**, and the **source** (BM25 vs. vector) they came from.
- **Reranker scores** and the **reranked order**.
- **Prompt template + version** and the **final prompt**.
- **Model name + configuration**, the **generated response**, and **citations**.
- **Input / output / total tokens** and the **cost** derived from them.
- **Per-stage latency** and **end-to-end latency**.
- **Errors, retries, fallbacks, and validation results** (e.g. did citation enforcement pass; did the reranker fall back to local).



5.4 Choosing a tracing platform

The candidates are **Langfuse**, **LangSmith**, and **Braintrust**. `prod-rag` uses **Langfuse**.

Platform	Model	Why it fits here
Langfuse	Open-source, self-hostable	Full control, no data leaves your box, free to run locally; strong tracing + cost tracking. Chosen.
LangSmith	Hosted (LangChain)	Excellent, but hosted-first and tied to the LangChain ecosystem, which this repo deliberately avoids.
Braintrust	Hosted eval-first	Great eval UX; hosted, and eval here is already covered by Ragas + CI.

Interview framing. “Why Langfuse?” “I wanted traces and cost/latency dashboards without shipping request data to a third party, so a self-hostable, open-source platform won. The tracing wrapper is defensive — it no-ops if `LANGFUSE_*` is unset — so the pipeline never depends on the tracer being up.”

The wrapper targets the **Langfuse v3/v4 SDK** (`start_as_current_span`, token usage folded into span metadata) and is a **full no-op when unconfigured**, so tests and offline runs never require a running collector.

5.5 Standing up Langfuse locally

```
docker compose -f docker-compose.langfuse.yml up -d # postgres + clickhouse + redis + minio + web + wor
# open http://localhost:3000, create a project, copy the keys into .env:
# LANGFUSE_PUBLIC_KEY=pk-lf-...
# LANGFUSE_SECRET_KEY=sk-lf-...
# LANGFUSE_HOST=http://localhost:3000
python -m rag.generate.answer "What is Azure Container Apps?" # traces now appear
```

REQUIRES YOU — live trace screenshots. The trace timeline and dashboard screenshots for the portfolio are produced by running the stack above and asking a few questions. This is a manual step (Docker + a few queries); the code and compose file are ready.

5.6 The metrics rollout

Independently of Langfuse, every request appends a line to `.metrics/requests.jsonl`, and `observability/metrics.py` rolls them up into percentiles, cost, citation coverage, and failure rate:

```
python -m rag.observability.metrics
```

Cost is computed from token counts at `gpt-4o-mini` pricing (\$0.15 / 1M input tokens, \$0.60 / 1M output tokens).

5.7 Measured operational results

MEASURED — rollup over 8 grounded queries against the Azure-docs corpus.

p50 latency	p90 latency	avg cost / request	citation coverage	failure rate
2326 ms	3709 ms	\$0.00025	100%	0%

prod-rag — evaluation & observability

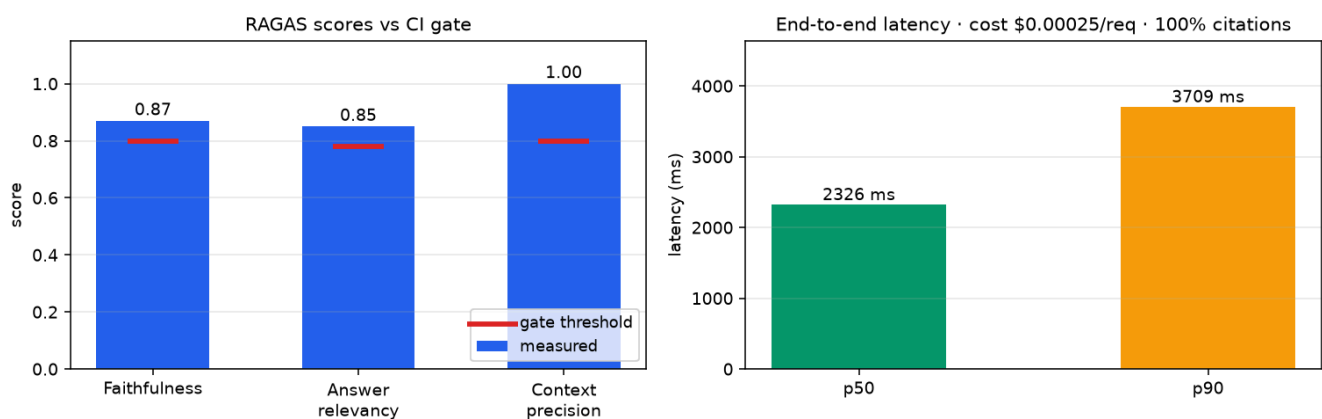


Figure 5.1 — Left: Ragas scores sit above their CI gate thresholds. Right: measured p50/p90 latency. Cost per request is \$0.00025 with 100% citation coverage.

Reading the numbers:

- **p50 2.3 s / p90 3.7 s** — the tail (p90) is ~1.6× the median, driven by generation time, not retrieval; retrieval and rerank are milliseconds against the LLM's seconds.
- **\$0.00025 per request** — with `gpt-4o-mini` and tight, reranked context, answers cost a fraction of a cent.

- **100% citation coverage, 0% failure** — every answer over this set carried valid citations, and none errored — the enforcement design (Chapter 4) made visible as an operational metric.

ESTIMATE — cost at scale. At \$0.00025/request, 10,000 requests/day $\approx$ **\$2.50/day** $\approx$ **\$75/month** in model spend (self-hosted retrieval and tracing add only infrastructure). Labelled an estimate because it extrapolates the measured per-request cost; real traffic mixes longer answers.

Note on p95/p99. The same `requests.jsonl` supports p95/p99, but with $n=8$ those percentiles are not statistically meaningful, so this book reports p50/p90 and shows the method rather than printing noisy tail numbers. With production traffic you would add p95/p99 straight from the same rollup.

5.8 The dashboards to build

Once the stack has traffic, chart these (Langfuse provides most out of the box):

- Latency percentiles (p50/p90/p95/p99) over time.
- Cost per request and daily/monthly projections.
- Citation coverage, faithfulness (sampled), context relevance.
- Failure rate, unsupported-answer rate, empty-retrieval rate, retry rate, model timeout rate.

Each chart should carry a caption stating what it shows and why it matters — a p90 spike is a user-experience story, not just a line.

5.9 A disciplined anomaly workflow

When a metric moves, follow the same loop every time (this is the *method*; the example is illustrative, not a claimed incident):

1. **What happened** — which metric changed, by how much, starting when?
2. **Which trace exposed it** — open a slow/failed request's trace; find the span that grew.
3. **Suspected root cause** — e.g. p90 rose because the reranker fell back to the local cross-encoder after a Cohere timeout, adding CPU time.
4. **Corrective action** — add a timeout+retry to the hosted reranker, or pin the local model warm.
5. **Result after the fix** — re-measure the same metric; confirm the tail returned to baseline.

EXAMPLE — anomaly note format (template, not a measured event).

```
Metric:    p90 latency 3.7s -> 6.1s
Trace:    req_9f2c - 'rerank' span 40ms -> 2.3s
Cause:    Cohere rerank timeout -> local fallback on cold CPU model
Action:    warm local reranker at startup; 1.5s timeout on hosted call
Result:    p90 back to 3.8s
```

5.10 The CI regression gate

The offline eval suite (Chapter 4) runs on every pull request via `.github/workflows/eval.yml`. It ingests the corpus, scores the golden set, and **fails the build if any metric drops below its threshold** in `config/retrieval.yaml` (faithfulness 0.80 / answer relevancy 0.78 / context precision 0.80). A subtle but important detail: a `nan` score (e.g. the judge failed) is treated as a **gate failure**, not a pass — “no signal” must never sneak through as “green.”

```
# .github/workflows/eval.yml (essence)
name: rag-eval-gate
on: [pull_request]
jobs:
  eval:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.11" }
      - run: pip install -e ".[dev]"
      - env: { OPENAI_API_KEY: ${ secrets.OPENAI_API_KEY } }
        run: python eval/run_ragas.py      # exits non-zero if below threshold
```

REQUIRES YOU — activate the gate. Set the CI secret so the judge can run: `gh secret set OPENAI_API_KEY --repo mimuruth/prod-rag`. Until then the job *skips* (green-because-skipped); with it, the job *enforces* (green-because-passed) — a much stronger signal, and worth doing before you show the repo.

EXAMPLE — the gate passing vs. blocking a PR.

```
# passing
faithfulness      0.87 >= 0.80 OK
answer_relevancy 0.85 >= 0.78 OK
context_precision 1.00 >= 0.80 OK
eval gate: PASS

# blocking (a prompt change regressed grounding)
faithfulness      0.71 < 0.80 FAIL
eval gate: FAIL -> pull request blocked by branch protection
```

This is the payoff of Chapters 3–5 together: a change that *looks* fine but *measurably* lowers answer quality cannot reach `main`.

References

- Langfuse, LangSmith, and Braintrust documentation; OpenTelemetry concepts (spans/traces); Google SRE Book (the “Four Golden Signals” and percentile monitoring); Ragas documentation.

Chapter 6 — Project 2: Offline Local SLM Lab

Repository: `local-slm-lab` · **What you will build:** a fully offline assistant on Ollama, a rigorous benchmark harness that compares three models on identical hardware and prompts, a temperature/determinism study, and a schema-constrained extraction path — all with **no API keys**.

6.1 Why run a model locally at all

Cloud LLMs are convenient, but real constraints rule them out:

- **Privacy** — data cannot leave the premises (healthcare, legal, government).
- **Regulatory compliance** — some regimes forbid sending data to third parties.
- **Latency** — a network round-trip is too slow or too jittery for the use case.
- **Cost at scale** — per-token pricing becomes prohibitive at high volume.
- **Edge deployment** — connectivity is not guaranteed.
- **Data sovereignty** — data must remain in a jurisdiction you control.

A **small language model (SLM)** — a few billion parameters — is often “good enough” for a bounded task and can run on a laptop. This project quantifies exactly what “good enough” costs in tokens/second and memory.

6.2 Setup and models

```
git clone https://github.com/mimiruth/local-slm-lab.git && cd local-slm-lab
ollama pull llama3.2 && ollama pull phi4-mini && ollama pull mistral
python -m venv .venv && .\.venv\Scripts\Activate.ps1
pip install -e "[dev]"
```

The three models span a useful size range: **Llama 3.2 (3B)**, **Phi-4 Mini (~4B)**, and **Mistral (7B)** — served at their default quantization by Ollama.

6.3 The application: CLI and API

The project ships both interfaces over the same core:

```
python src/cli.py ask "Summarize: the quarterly report shows 12% revenue growth."
python src/cli.py extract "Invoice from Contoso Ltd. Amount due: $1,240.50 USD."
uvicorn api:app --reload # the FastAPI wrapper
```

6.4 Benchmark methodology

A benchmark is only meaningful if everything except the variable under test is held constant.

`bench/runner.py` fixes hardware, the prompt set (`prompts/standard_set.jsonl`), generation settings, and context size, and varies only the model. It measures:

- **Time to first token (TTFT)** — responsiveness; when the user first sees text.
- **Tokens per second** — sustained decode throughput (`eval_count / eval_duration` from Ollama).
- **Total latency** — full response time.
- **Model-loading (cold-start) time** — measured *separately* via a warm-up pass.
- **Peak memory** — resident set while generating.

The warm-vs-cold distinction (the key methodological idea). The first request to an idle model pays a one-time *model-load* penalty. `runner.py` does a **warm-up pass first**, records its first-response latency as **cold start**, then times steady-state **warm** requests. Reporting a single blended number would hide the single most important production decision: keep the model resident, or pay the load penalty on every cold request?

Run it:

```
python -m bench.runner --models llama3.2 phi4-mini mistral
```

6.5 Measured benchmark results

MEASURED — Intel Core Ultra 9 285H · 63 GB RAM · RTX 5070 Laptop GPU · Ollama 0.32.5 · Windows · warm steady-state; cold start = first response including model load.

Model	Params	Mem (GB)	Tokens/sec	Warm TTFT (ms)	Total (ms)	Cold start (ms)
llama3.2	3B	2.55	89.6	455	2841	3008
phi4-mini	~4B	3.09	69.6	534	3113	2632
mistral	7B	4.95	53.1	126	2818	8414

local-slm-lab — offline model comparison

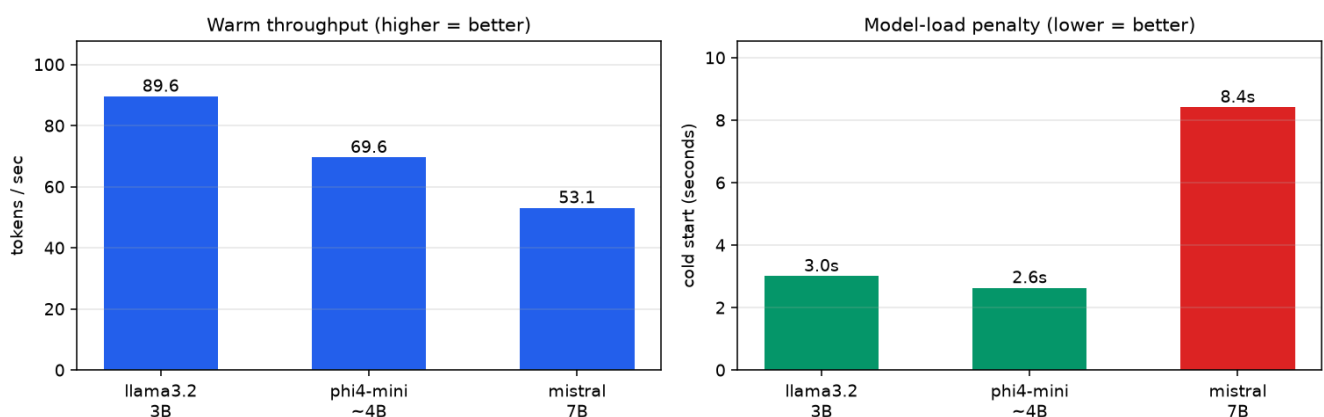


Figure 6.1 — Warm throughput (left) and cold-start penalty (right). Llama 3.2 decodes fastest; Mistral 7B pays an 8.4 s model-load penalty.

Reading the numbers:

- **Fastest throughput:** `llama3.2` at **89.6 tok/s** — the smallest model decodes fastest, as expected.
- **Lowest warm TTFT:** `mistral` at **126 ms** once loaded — first-token latency and sustained throughput are *independent axes*; the 7B model starts emitting quickly but then decodes slowest.
- **Cold start scales with size:** 2.6 s (`phi4-mini`) → 3.0 s (`llama3.2`) → **8.4 s (mistral 7B)**. Bigger weights take longer to load. This is the deployment tradeoff: a 7B model that idles out of memory pays 8 s on the next request.
- **Memory tracks parameters:** 2.5 → 3.1 → 5.0 GB.

Takeaways. For latency-sensitive, always-warm serving, `llama3.2` gives the best throughput-per-GB. For quality-per-token where a heavier model helps, keep `mistral` **pinned in memory** (`OLLAMA_KEEP_ALIVE`) to avoid its 8 s cold start.

6.6 Temperature and determinism study

`bench/temp_study.py` runs each prompt 5× at temperature 0.0 and 0.7 and counts **distinct outputs** (via SHA-256 of the trimmed text) and **valid-JSON count**.

```
python -m bench.temp_study --model llama3.2 --runs 5
```

MEASURED — llama3.2 , 5 runs per prompt.

Temp	Distinct outputs / 5 runs	Valid JSON / 5	Behaviour
0.0	1 / 5 (every prompt)	0 / 5	fully deterministic — reproducible
0.7	4–5 / 5	0 / 5	high variance — diverse

Two lessons:

1. **Temperature 0 is fully deterministic here** — 1 distinct output across 5 runs for every prompt. Essential where you need reproducibility (extraction, tool calls). Temperature 0.7 diversifies sharply (4–5 distinct/5) — good for brainstorming, risky for structured output.
2. **Determinism ≠ parseability.** Even at temperature 0, *plain prompting produced 0/5 valid JSON* — the model wrapped JSON in prose or markdown fences. That motivates the next section.

6.7 Structured output that you can trust

Plain prompting cannot be relied upon to emit valid JSON (the study proves it). `schema/extract.py` enforces a **Pydantic** schema with **Instructor**, which validates the model's output and **retries once** on failure, against Ollama's OpenAI-compatible endpoint.

MEASURED — same 3B model, before vs. after. Plain prompting: **0/5 valid JSON**. With Instructor + Pydantic (validate + retry-once): a validated, typed object every time.

```
$ python src/cli.py extract "Invoice from Contoso Ltd. Amount due: $1,240.50 USD, due 2026-09-01." --mode
{
  "vendor": "Contoso Ltd.",
  "total": 1240.5,
  "currency": "USD",
  "due_date": "2026-09-01"
}
```

Interview framing. “How did you get reliable JSON from a small local model?” “I measured that plain prompting gave 0/5 valid JSON even at temperature 0, so I stopped hoping and *enforced* a Pydantic schema with Instructor — it validates and retries on invalid output. The reliability comes from the validation loop, not from the model’s goodwill.”

6.8 Quantization and the quality-vs-speed tradeoff

Ollama serves quantized weights by default (e.g. `Q4_K_M`), which is why a 7B model fits in ~5 GB (Section 6.5). Lower-bit quantization trades a little quality for less memory and faster load. The honest extension of this project is a quantization sweep (`Q4_K_M` vs `Q5` vs `Q8`) recording the quality-vs-speed delta on a 30–50 prompt set with a 0–5 quality rubric — the harness is built for exactly this; the current results use the default quantization and a 3-prompt starter set, which is stated plainly in `results/benchmark.md`.

6.9 Reproduce and record

```
python -m bench.runner --models llama3.2 phi4-mini mistral # -> results/benchmark.md
python -m bench.temp_study --model llama3.2 --runs 5 # -> results/temp_variance.md
python scripts/plot_results.py # -> docs/benchmark.png
make test
git add results/ docs/ && git -c user.name="mimiruth" -c user.email="mimiruth@users.noreply.github.com" c
```

The raw result files, summary tables, chart, and hardware caveats all live in the repo, so anyone can reproduce the experiment and get comparable numbers on their own hardware.

References

- Ollama documentation and model library; Instructor (`python.useinstructor.com`); Pydantic documentation; model cards for Llama 3.2, Phi-4, and Mistral; `llama.cpp` quantization notes (GGUF / K-quants).

Chapter 7 — Project 4: Fine-Tuning with LoRA, QLoRA, and DPO

Repository: `llm-finetuning` · **What you will build:** a parameter-efficient fine-tuning pipeline that takes a base model, teaches it a *skill* (structured JSON extraction) with LoRA, aligns it further with DPO, and reports before/after metrics — runnable end-to-end on a GPU with **one command**.

REQUIRES YOU — a CUDA GPU. This is the one project this book cannot run on the reference laptop. Everything is implemented and the run is a single script (Section 7.9); the before/after numbers are the book's one **REQUIRES YOU** result set, awaiting a GPU box (a few dollars on a rented instance).

7.1 The training vocabulary

- **Pretraining** — the original, enormous next-token training that produces a *base* model. You will not do this; it costs millions.
- **Instruction tuning / Supervised Fine-Tuning (SFT)** — continue training on (*prompt, ideal response*) pairs so the model follows instructions in a format.
- **Parameter-Efficient Fine-Tuning (PEFT)** — train a *tiny* number of new parameters instead of all billions. LoRA is the dominant method.
- **LoRA (Low-Rank Adaptation)** — freeze the base weights; inject small low-rank matrices (adapters) into each layer and train only those. The update is $W + BA$ where B and A are low-rank, so you train megabytes, not gigabytes.
- **QLoRA** — LoRA on top of a **4-bit quantized** base model, so a model that would need ~32 GB in full precision trains in ~16 GB. This is what makes a 4B model tunable on a single consumer GPU.
- **Adapters** — the small trained LoRA weights; portable, stackable, swappable without touching the base model.
- **Preference data** — triples of (*prompt, chosen, rejected*) responses that express which answer is better.
- **Direct Preference Optimization (DPO)** — train directly on preference triples to make *chosen* more likely and *rejected* less, **without a separate reward model** (the simpler successor to RLHF).
- **Reward modelling / RLHF** — the older two-stage alignment (train a reward model, then RL against it); DPO collapses this into one stable step.
- **Alignment** — shaping *behaviour* (helpfulness, refusal, format) rather than adding knowledge.
- **Catastrophic forgetting** — over-training on a narrow task erodes general ability. PEFT + modest epochs mitigate it.
- **Overfitting** — memorising the training set; caught by a held-out validation split.
- **Evaluation leakage** — test examples (or near-duplicates) leaking into training, inflating scores. Prevented by deduplicating across splits.

7.2 Choosing a measurable task

Fine-tune for a **skill or format**, not for facts (Section 1.6). The task here is **structured JSON extraction from messy text** (with tool-call selection and correct refusal as siblings), because success is *objectively measurable*:

- **JSON validity rate** — does the output parse?
- **Schema compliance / field-level accuracy** — are the right fields present and correct?
- **Exact-match accuracy** — does the object equal the gold object?
- **Refusal correctness** — does it refuse when it should?

Contrast with “make it a better writer,” which has no crisp metric. A fine-tuning project lives or dies on whether you can *score* it.

7.3 The dataset pipeline

`data/prepare.py` builds two datasets:

- **SFT** from `HuggingFaceH4/ultrachat_200k` (instruction-following chat).
- **DPO** from `argilla/distilabel-intel-orca-dpo-pairs` (preference triples).

The full pipeline you should apply (and that the code scaffolds):

1. **Source selection + licensing** — prefer permissively licensed datasets; record the license.
2. **Inspection → filtering → deduplication** — remove malformed, too-long, or duplicate rows; dedup *across* splits to prevent leakage.
3. **Normalisation + schema validation** — enforce the target output schema on training targets so you never train on invalid gold.
4. **Splits** — train / validation / test, disjoint.
5. **Edge cases + refusal examples** — include inputs that *should* be refused, so the model learns to abstain.
6. **Preference pairs** — for DPO, construct (*prompt, chosen, rejected*) where `chosen` is valid/correct and `rejected` is a plausible-but-wrong output.

```
python data/prepare.py --task both --limit 2000
```

Why a task-specific set even when general sets exist. General instruction data teaches broad following; it will not make the model reliably emit *your* JSON schema or refuse *your* out-of-scope inputs. General data is the foundation; a small, high-quality task set is what moves the target metric. *Quality >> quantity* — 2k excellent examples beat 200k noisy ones.

7.4 TRL vs. Axolotl

Tool	Style	Use when
Hugging Face TRL	Python scripts (<code>SFTTrainer</code> , <code>DPOTrainer</code>)	You want explicit control and to read exactly what runs. Primary here.
Axolotl	One declarative YAML config	You want a fast, batteries-included run and are happy to configure via YAML. Provided as the alternative (<code>configs/sft.yaml</code> , <code>configs/dpo.yaml</code>).

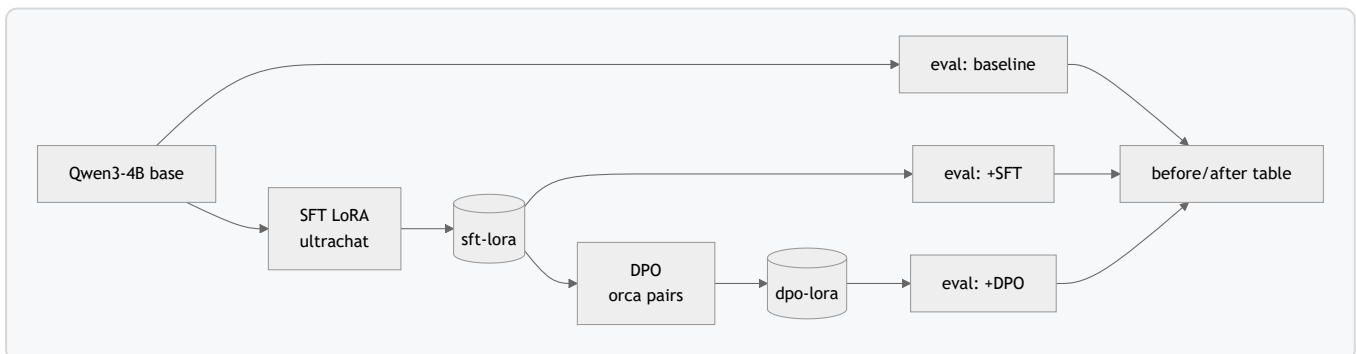
The repo ships **both**: TRL scripts (`scripts/train_sft.py` , `scripts/train_dpo.py` , `scripts/eval.py`) for control, and Axolotl configs for a one-file run.

7.5 Configuration and hyperparameters

Base model: `Qwen/Qwen3-4B` . The knobs that matter, and how to think about them:

Hyperparameter	Typical	What it controls
Quantization	4-bit (QLoRA)	Base-model memory during training
LoRA rank <code>r</code>	8–32	Capacity of the adapter (bigger = more to learn, more to overfit)
LoRA alpha	16–64	Scaling of the adapter update (often $2 \times r$)
LoRA dropout	0.05	Regularisation on the adapter
Batch size	small	Limited by VRAM
Gradient accumulation	4–16	Simulates a larger batch without more VRAM
Learning rate	$1e-4$ – $2e-4$	Step size (higher than full fine-tuning)
Epochs	1–3	Too many → overfitting / forgetting

Training is **stackable**: base → SFT adapter → DPO adapter *on top of* the SFT adapter.



7.6 Training commands

```
# TRL path
python scripts/train_sft.py --base-model Qwen/Qwen3-4B          # -> artifacts/sft-lora
python scripts/train_dpo.py --base-model Qwen/Qwen3-4B --adapter artifacts/sft-lora # -> artifacts/dpo-l

# Axolotl alternative
accelerate launch -m axolotl.cli.train configs/sft.yml
accelerate launch -m axolotl.cli.train configs/dpo.yml
```

Record **training curves** (loss over steps) with your experiment tracker; a healthy SFT run shows loss falling then plateauing. Checkpointing lets you resume an interrupted run.

7.7 Evaluation: base vs. SFT vs. SFT+DPO

`scripts/eval.py` loads the base model with an optional adapter and scores it with `eval/metrics.py` (`json_validity_rate`, `exact_match_rate`, `refusal_correctness`) over a held-out extraction set (`data/test_extraction.jsonl`):

```
python scripts/eval.py --model Qwen/Qwen3-4B          # baseline
python scripts/eval.py --model Qwen/Qwen3-4B --adapter artifacts/sft-lora
python scripts/eval.py --model Qwen/Qwen3-4B --adapter artifacts/dpo-lora
```

REQUIRES YOU — the before/after table. Run the script above on a GPU and fill this in. The shape is fixed; the numbers are collected by *you*:

Metric	Base	+ SFT	+ DPO
JSON validity rate	—	—	—
Exact-match accuracy	—	—	—
Refusal correctness	—	—	—

This book will **not** print invented numbers here. Everything up to the GPU boundary is implemented and runnable.

What to expect and watch for (interview-ready)

- **SFT should raise JSON validity and exact-match** most — it directly teaches the format.
- **DPO should add incremental gains** on the softer preferences (cleaner refusals, fewer hallucinated fields). If DPO does *not* help, that is a valid, honest finding — it usually means the preference pairs were not discriminative enough, and the fix is better pairs, not more epochs.
- **Watch for:** invalid outputs (schema too strict, or LR too high destabilised the model), overfitting (train loss down, val metrics flat), and leakage (suspiciously perfect exact-match → check dedup across splits).

7.8 Deciding QLoRA vs. full fine-tuning

Interview framing — “Why QLoRA, not full fine-tuning?” “Full fine-tuning a 4B model needs many tens of GB of VRAM and produces a full-size checkpoint per experiment. QLoRA trains 4-bit base + small adapters in ~16 GB and yields a few-MB adapter I can version and swap. For a single-task skill, LoRA reaches comparable quality at a fraction of the cost — the pragmatic default.”

7.9 One command on a GPU box

```
pip install torch --index-url https://download.pytorch.org/whl/cu121 # match your CUDA
bash scripts/run_all.sh
```

`run_all.sh` chains the whole pipeline: install → baseline eval → data prep → SFT → eval → DPO → eval, printing the metrics you paste into Section 7.7. Adapters are pushed to the Hugging Face Hub (they are git-ignored), keeping the repo lean.

References

- Hu et al. (2021), “LoRA”; Dettmers et al. (2023), “QLoRA”; Rafailov et al. (2023), “Direct Preference Optimization”; Hugging Face TRL and PEFT documentation; Axolotl documentation; Qwen3 model card.

Chapter 8 — Project 5: Real-Time Multimodal (Voice)

Repository: `realtime-voice` · **What you will build:** a streaming **ASR** → **LLM** → **TTS** voice assistant with a per-stage **latency budget**, timeouts and fallbacks, a **replay mode** for deterministic debugging, and failure injection — measured end-to-end.

8.1 Batch vs. streaming: a different world

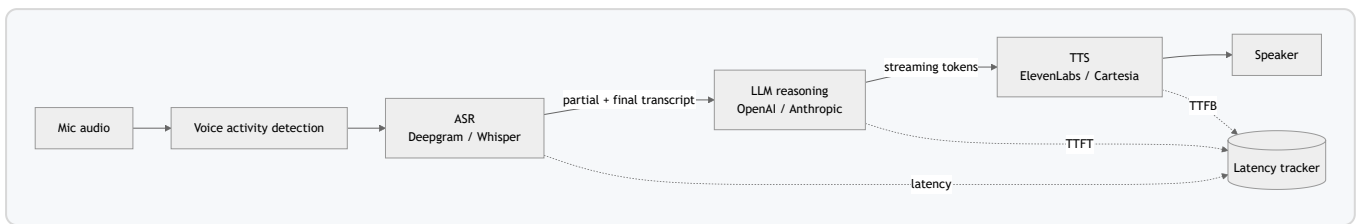
Projects 1–4 are request/response: send input, wait, get output. A voice assistant is **streaming** — audio arrives continuously and the user is *waiting to hear a reply*, so every millisecond of every stage is on the critical path.

The streaming vocabulary you must know:

Term	Meaning
Streaming data	Continuous input (audio frames), processed as it arrives
Event-driven	Components react to events rather than polling
WebSockets	Full-duplex, persistent connection — used to orchestrate the pipeline
Server-Sent Events (SSE)	One-way server→client stream (e.g. token streaming)
Backpressure	Slowing a fast producer when a consumer can't keep up
Bounded queues	Fixed-size buffers that make backpressure explicit
Load shedding	Dropping work under overload to protect latency
Flow control	Governing how fast data moves between stages
Timeouts / cancellation	Bounding how long a stage may take; stopping stale work
Retries / circuit breakers	Retrying transient failures; cutting off a failing dependency
Partial failure	One stage fails while others are healthy
Graceful degradation	Delivering a reduced-but-useful response instead of nothing
Replay	Re-running recorded inputs deterministically
Idempotency / ordering / dedup	Correctness under retries, reordering, and duplicate events
Session management	Tracking one conversation's state across turns

The core idea. In a real-time system, *being late is the same as being wrong*. A perfect reply that arrives after the user gives up is a failure. So we **measure per stage** and **degrade gracefully** rather than block.

8.2 The voice pipeline



Stages and the hard parts of each:

- **Audio capture → VAD** — detect when the user is actually speaking, so you don't send silence to ASR.
- **ASR partial vs. final transcripts** — partials let downstream start early; the final is authoritative.
- **LLM reasoning, streaming tokens** — begin TTS on the first tokens rather than waiting for the whole reply.
- **TTS → playback** — synthesise and start audio as bytes arrive (time-to-first-byte matters more than total).
- **Interruption / barge-in** — if the user speaks while the assistant is talking, *cancel* the current turn and listen. This is what makes it feel real.
- **Session recovery** — a dropped WebSocket must not lose the conversation.

8.3 Architecture and providers

`src/pipeline/orchestrator.py` defines a `Pipeline` of `StageSpec` `s`. `build_demo_pipeline()` wires **mock** stages (deterministic, no keys); `build_pipeline()` swaps in **real** providers for any whose API key is set and falls back to a mock otherwise — so the *same code path* runs with zero, some, or all keys configured.

Role	Providers
ASR	Deepgram / Whisper
LLM	OpenAI / Anthropic
TTS	ElevenLabs / Cartesia
Orchestration	WebSockets (Pipercat-style)

Alternative designs discussed. The same skeleton supports a **computer- vision assistant** (frames → detector → LLM → overlay) or a **streaming log analyzer** (log lines → windowed features → LLM → alerts). Voice was chosen because it exercises the tightest latency budget and the most user-visible failure modes (barge-in, dead air).

8.4 Setup

```
git clone https://github.com/mimiruth/realtime-voice.git && cd realtime-voice
python -m venv .venv && .\.venv\Scripts\Activate.ps1
pip install -e ".[dev]"
python -m pipeline.orchestrator # offline demo (mock stages, no keys)
```

8.5 The latency budget

The whole discipline of real-time is **budgeting** the delay. `latency/tracker.py` records per-stage timings and aggregates p50/p90 across requests.

MEASURED — offline demo baseline (mock stages, real instrumentation), 5 requests through the pipeline. The mock stages produce deterministic timings that validate the tracker end to end; with real provider keys the *same tracker* emits real provider latencies.

Stage	Metric	p50	p90
ASR	transcript	61.8 ms	63.1 ms
LLM	response	93.0 ms	93.3 ms
TTS	audio bytes	46.9 ms	48.4 ms
End-to-end	mouth-to-ear	~202 ms	~205 ms

realtime-voice — per-stage latency budget (p50)

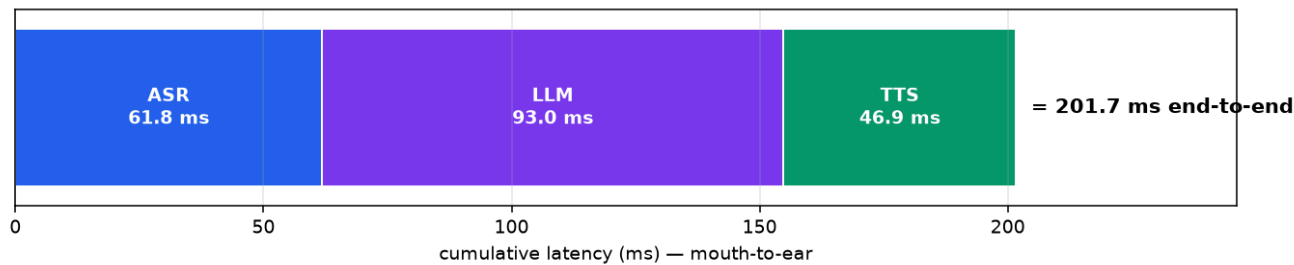


Figure 8.1 — The latency waterfall. The stages sum to the ~202 ms the user perceives before hearing a reply; the LLM stage dominates the budget.

Honesty note. These are the *instrumentation baseline* from mock stages, not live provider numbers — stated plainly so no one mistakes them for a Deepgram/ ElevenLabs benchmark. What is proven is that the per-stage measurement, percentile aggregation, and budget accounting work end to end.

The full budget you measure once live includes: audio buffering, ASR time-to-partial and time-to-final, LLM TTFT and generation duration, TTS time-to-first-byte, playback startup, and network/serialization overhead. Chart each request as a **waterfall** and the fleet as **percentile** curves.

8.6 Resilience: timeouts, fallbacks, degradation

`resilience/fallback.py` provides `with_timeout`, and every stage is wrapped so a slow or failed stage **degrades gracefully** instead of blocking:

- **Per-service timeouts + retry limits** — no stage may hang the turn.
- **Cancellation** — barge-in cancels the in-flight turn immediately.
- **Fallback models / text fallback** — if TTS fails, return text; if the primary LLM times out, use a secondary or a canned acknowledgement.
- **Circuit breakers** — after repeated failures, stop calling a dead dependency for a cool-off period instead of timing out every request.
- **Backpressure + bounded queues + load shedding** — under overload, drop the oldest frames rather than growing latency without bound.
- **User-facing messages** — never dead air; acknowledge a delay openly.

Design principle. Every external call has three outcomes planned in advance: success, slow (timeout → fallback), and failure (circuit-break → degrade). If you have not decided what happens on “slow,” you have not finished the feature.

8.7 Replay mode and failure injection

`replay/recorder.py` records inputs and can replay them deterministically — invaluable for reproducing a failed session without a live microphone:

```
python -m replay.recorder --play sessions/example.jsonl
```

Failure-injection testing deliberately breaks each dependency to verify the fallback:

Injected failure	Expected user experience
ASR failure	“Please repeat” prompt; secondary ASR if configured
LLM timeout	Spoken acknowledgement of delay, never silence
TTS failure	Text fallback of the reply
WebSocket disconnect	Session recovers; conversation state preserved
Malformed / duplicate / out-of-order events	Rejected / deduped / reordered — no crash
Queue saturation / network delay	Load-shedding keeps latency bounded
Dependency outage	Circuit breaker opens; degraded mode

REQUIRES YOU — live runs. The instrumentation, resilience, and replay are real and testable offline. *Live* latency and live failure behaviour need `DEEPGRAM_API_KEY`, `OPENAI_API_KEY` / `ANTHROPIC_API_KEY`, and `ELEVENLABS_API_KEY` in `.env`; add any subset and that stage runs live while the rest stay mocked.

8.8 Run it

```
python -m pipeline.orchestrator          # offline demo
python scripts/plot_results.py           # -> docs/latency-budget.png
make test
# live (with keys):
cp .env.example .env                    # paste the keys you have
python -m pipeline.orchestrator
```

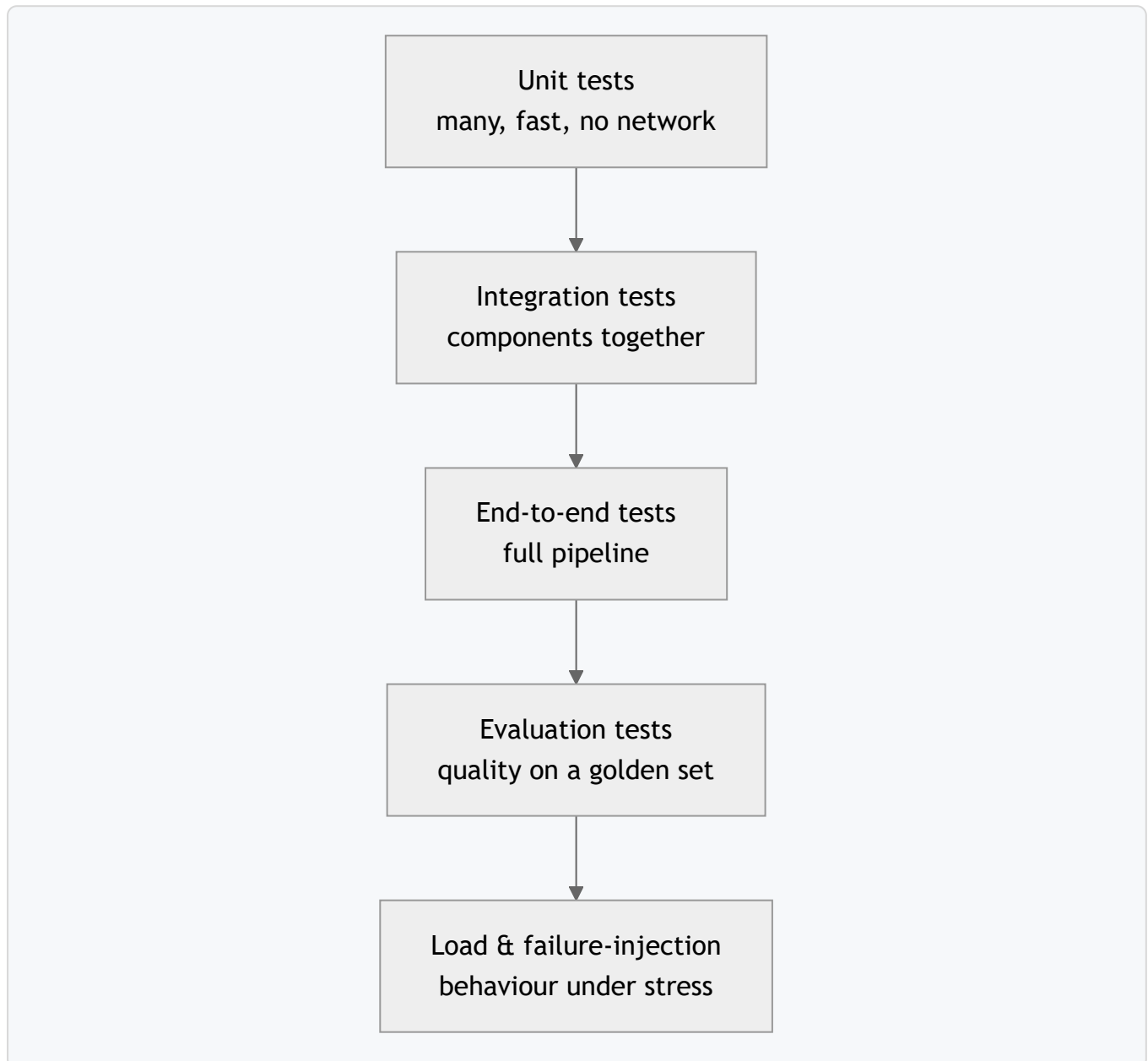
References

- Pipecat documentation; Deepgram, OpenAI, Anthropic, ElevenLabs, and Cartesia API docs; Nygard, “Release It!” (circuit breakers, bulkheads); the WebSocket protocol (RFC 6455); “Lost in the Middle” (context) as it applies to streaming reasoning.

Chapter 9 — Testing Strategy

Tests are how you make change *safe*. Across the four repositories the philosophy is the same: fast unit tests on every push, heavier checks gated behind CI, and — uniquely for AI systems — **evaluation as a test** (Chapters 4–5). This chapter is the catalogue.

9.1 The testing pyramid, adapted for AI systems



Classic software has the top four layers. AI systems add **evaluation tests** — quality is a *measured property*, not a pass/fail assertion, so it needs its own layer (the Ragas gate in Chapter 5, the benchmark harness in Chapter 6).

9.2 The suites

For each suite: *what* it tests, *why* it matters, *how* to run it, and what pass looks like.

Unit tests

- **What / why:** individual pure functions (chunker, RRF fusion, citation parsing, metric percentiles) — fast, deterministic, no network.
- **How:** `pytest` (each repo). Example real tests: `prod-rag test_chunker.py`, `test_citations.py`, `test_metrics.py`; `local-slm-lab test_prompts.py`, `test_temp_study.py`.
- **Pass:** all green in seconds; a broken invariant (e.g. overlap math) fails immediately.

Integration tests

- **What / why:** components wired together — e.g. hybrid retrieval over a real in-memory index, or the Instructor extraction path against a local model.
- **How:** `pytest` with a small fixture corpus/model.
- **Pass:** the pieces produce the expected joined behaviour (retrieve → rerank returns on-topic passages).

End-to-end tests

- **What / why:** the whole pipeline for one input (`answer()` returns a grounded answer or a clean abstention).
- **How:** run the pipeline over a fixed question; assert shape (`answer`, `citations`, `refused`).
- **Pass:** in-corpus question → grounded answer with citations; out-of-corpus → `refused: true`.

Regression / evaluation tests (the AI-specific one)

- **What / why:** quality must not silently drop. The Ragas gate scores the golden set and **fails the build** below threshold (Chapter 5).
- **How:** `python eval/run_ragas.py` locally; automatically on every PR.
- **Pass:** faithfulness ≥ 0.80 , answer relevancy ≥ 0.78 , context precision ≥ 0.80 . A `nan` counts as failure.

Load tests

- **What / why:** latency under concurrency; find the point where p90 degrades.
- **How:** drive the FastAPI endpoint with a load tool (e.g. `locust` / `k6`) at rising concurrency; watch p50/p90 from the metrics rollout.
- **Pass:** latency stays within budget up to the target concurrency; degradation is graceful, not a cliff.

Failure-injection tests

- **What / why:** verify fallbacks (Chapter 8). Break ASR/LLM/TTS/WebSocket and confirm the user still gets a useful, timely response.
- **How:** `realtime-voice` injects failures via the mock stages + replay.
- **Pass:** each injected failure yields the *documented* degraded behaviour, not a crash or dead air.

Contract & schema-validation tests

- **What / why:** external I/O keeps its shape. The extraction output must satisfy its Pydantic schema; the API response must match its contract.
- **How:** Pydantic validation in code + tests that assert schema conformance.
- **Pass:** malformed output is rejected/retried, never silently accepted.

Prompt tests

- **What / why:** prompts are versioned code (Chapter 4); a prompt change is evaluated, not merged on vibes.
- **How:** the eval gate *is* the prompt test — a prompt edit that lowers faithfulness fails CI.
- **Pass:** prompt changes hold or improve the gated metrics.

Retrieval tests

- **What / why:** the retriever returns the right passages for known queries.
- **How:** assert that a query for a known fact retrieves the chunk containing it.
- **Pass:** the supporting chunk appears in the top- `n` .

Security tests

- **What / why:** no secret ever lands in git; no known-vulnerable dependency ships.
- **How:** `gitleaks detect` + pre-commit (Chapter 3), `pip-audit` / Dependabot.
- **Pass:** gitleaks reports no leaks; audit reports no criticals.

9.3 Running everything

```
make test                # unit + integration (each repo)
python eval/run_ragas.py  # evaluation gate (prod-rag)
python -m bench.runner ... # benchmark (local-slm-lab)
gitleaks detect --source . # security
pre-commit run --all-files # hooks (lint + gitleaks)
```

9.4 Reading a test summary

EXAMPLE — a consolidated status (format; fill with your run).

Suite	Total	Pass	Fail	Skip	
unit + integration	9	9	0	0	
evaluation gate	3	3	0	0	(faithfulness 0.87, ...)
gitleaks	-	-	0	-	no leaks
build status: GREEN					

Code coverage is reported by `pytest --cov` ; the exact percentage is a number *you* generate on your machine, so it is left as a slot rather than invented.

9.5 Troubleshooting the common failures

Symptom	Likely cause	Fix
<code>ModuleNotFoundError: rag</code>	<code>src</code> not on path	run with <code>PYTHONPATH=src</code> or <code>pip install -e .</code>
Eval gate <code>nan</code>	judge/network error or empty retrieval	check <code>OPENAI_API_KEY</code> , ensure indexes were built
Ollama tests hang	server not running	start Ollama; <code>curl localhost:11434/api/tags</code>
gitleaks flags a test fixture	example key in a sample file	move to <code>.env.example</code> with an empty value
Flaky latency assertions	timing on a busy machine	assert budgets, not exact ms; use percentiles

Tests give you the confidence to deploy — which is Chapter 10.

References

- [pytest documentation](#); [Ragas](#); [locust / k6 load-testing docs](#); [Nygard, “Release It!”](#) (stability testing); [OWASP testing guidance](#); [gitleaks](#) & [pip-audit](#).

Chapter 10 — Deployment and Shipping

“Runs on my laptop” is a prototype. “Anyone can pull an image and run it, and a tagged release deploys itself” is a product. This chapter takes `prod-rag` from local code to a self-publishing container, and surveys the deployment options and their tradeoffs for all five projects.

10.1 Containerising with Docker

- **What a container is:** your app plus its exact dependencies, frozen into a portable image that runs identically anywhere Docker runs.
- **Why it exists:** to kill “works on my machine” — the image *is* the machine.
- **Where it fits:** it is the deployable unit for every cloud target below.

`prod-rag` ships a `Dockerfile` that installs the `[serve]` extra (FastAPI + Uvicorn), copies the source, prompts, config, and corpus, and on first boot builds the indexes then serves:

```
FROM python:3.12-slim
WORKDIR /app
ENV PYTHONPATH=/app/src PYTHONUNBUFFERED=1
COPY pyproject.toml ./
COPY src ./src
COPY api.py ./
COPY prompts ./prompts
COPY config ./config
COPY docs ./docs
RUN pip install --no-cache-dir -e ".[serve]"
EXPOSE 8000
CMD ["sh", "-c", "[ -d .chroma ] || python -m rag.ingest.loaders --source docs/; uvicorn api:app --host 0
```

The FastAPI wrapper (`api.py`) exposes `GET /healthz` (liveness) and `POST /ask` .

Build and run locally:

```
docker build -t prod-rag:local .
docker run -p 8000:8000 -e OPENAI_API_KEY=sk-... prod-rag:local
curl -X POST localhost:8000/ask -H "content-type: application/json" -d '{"question":"What is Azure Contai
```

10.2 Health checks and readiness

- **Liveness (`/healthz`)** — “is the process up?” If it fails, restart the container.
- **Readiness** — “is it ready to serve?” The `prod-rag` container builds indexes on first boot, so readiness should wait until the index exists before routing traffic. In Container Apps/Kubernetes you map these to health probes so a still-warming instance does not receive requests.

10.3 Docker Compose (multi-service)

prod-rag's `docker-compose.langfuse.yml` stands up the full observability stack (Postgres, ClickHouse, Redis, MinIO, Langfuse web + worker) with one command (Chapter 5). Compose is ideal for *local* multi-service development; for production you graduate to a managed platform or Kubernetes.

10.4 Registries and the self-publishing pipeline

A **container registry** stores images. `prod-rag` publishes to **GitHub Container Registry (GHC)** automatically: `.github/workflows/docker.yml` builds and pushes on any `vx.y.z` tag, using the built-in `GITHUB_TOKEN` (no secrets to configure), with semver + SHA tags and layer caching.

```
git tag -a v1.0.0 -m "v1.0.0 - production RAG + observability + CI gating"
git push origin v1.0.0                # -> triggers the build; image appears at
                                      #   ghcr.io/mimiruth/prod-rag:1.0.0 and :latest
```

MEASURED (repo state). `prod-rag` is tagged `v1.0.0`; the `docker` workflow is wired to publish on that tag. Making the resulting GHCR package **public** is a one-click manual step (Package settings → visibility).

10.5 Deploying to the cloud (Azure Container Apps)

With the image published, one command deploys it:

```
az containerapp up \
  --name prod-rag --resource-group rg-prod-rag --location eastus \
  --image ghcr.io/mimiruth/prod-rag:1.0.0 \
  --ingress external --target-port 8000 \
  --env-vars OPENAI_API_KEY=secretref:openai-key
```

REQUIRES YOU — an Azure subscription. The Dockerfile, image, and command are ready; the actual deploy needs your subscription and an `OPENAI_API_KEY` stored as a Container Apps secret. Add `LANGFUSE_*` env vars to stream live traces.

10.6 Secrets, configuration, logging, monitoring in production

- **Secrets:** never in the image. Inject at runtime as platform secrets (Container Apps secrets, Kubernetes Secrets, cloud secret managers).
- **Configuration:** environment variables (the same `.env` keys), not code.
- **Logging:** structured logs to stdout; the platform aggregates them.
- **Monitoring:** the Chapter 5 metrics + Langfuse traces; wire p90/cost/failure alerts.

10.7 Choosing a target: the tradeoff table

Target	Best for	Tradeoff
Local only	Dev, demos, air-gapped	No scaling, no availability guarantees
Azure Container Apps	Serverless containers, scale-to-zero	Less control than raw K8s; used here for <code>prod-rag</code>
AWS (Fargate/App Runner)	AWS-native serverless containers	Ecosystem lock-in
Google Cloud Run	Simple serverless containers	Similar tradeoffs to ACA
Kubernetes (AKS/EKS/GKE)	Full control, complex topologies	Highest operational burden
Serverless functions	Spiky, short, stateless work	Cold starts; not ideal for warm models
Dedicated VM	Full control, steady load	You own patching, scaling, HA
Managed AI services	Fastest path, no infra	Least control; data leaves your box
Self-hosted models	Privacy, cost at scale	You run the GPUs

Per project:

- `prod-rag` — a container on ACA/Cloud Run (stateless API + local index).
- `local-slm-lab` — the whole point is *not* the cloud; ship as a container or binary to the edge/on-prem, GPU optional.
- `llm-finetuning` — a **GPU** VM or managed training job; adapters to the HF Hub; the resulting model deploys like any other.
- `realtime-voice` — a stateful, WebSocket service; a container platform that supports long-lived connections (ACA, a VM, or K8s), *not* short-lived serverless functions.

10.8 Scaling, cost control, rollback, releases

- **Scaling:** stateless services (`prod-rag`) scale horizontally behind a load balancer; the model API is the bottleneck, so cache and cap concurrency.
- **Cost control:** scale-to-zero for spiky traffic; a smaller model or the local SLM (Chapter 6) where quality allows; the **measured** \$0.00025/request (Chapter 5) is your unit economics.
- **Rollback:** deployments reference an *immutable image tag*; roll back by pointing at the previous tag (`:1.0.0` ← `:0.9.0`). Never deploy `:latest` to production for exactly this reason.
- **Release management:** tag → CI builds image → deploy the tag → keep a changelog. GHCR + `docker.yml` make the first two automatic.

Interview framing — “How would you deploy and scale this?” “Containerise; publish an immutable, tagged image on every release via CI; deploy that tag to a serverless container platform with health probes and scale-to-zero; keep secrets in the platform’s secret store; and monitor p90/cost/failure with alerts. Roll back by re-pointing at the previous tag. For the fine-tuning and voice services I’d diverge — GPU jobs and long-lived WebSocket hosts respectively.”

References

- Docker & Docker Compose docs; GitHub Actions + GHCR docs; Azure Container Apps, AWS App Runner/Fargate, Google Cloud Run, and Kubernetes documentation; the Twelve-Factor App (config, logs, disposability).

Chapter 11 — Portfolio Readiness

“Implemented” and “portfolio-ready with real numbers” are different milestones. This chapter is the checklist that closes the gap, plus an honest scorecard of where the four repositories actually stand.

11.1 The readiness checklist (apply to every project)

#	Item	Why it matters to a reviewer
1	Clean repository structure	They can find things; signals discipline
2	Complete README	The 60-second judgement is made here
3	Architecture diagram	Shows you think in systems, not scripts
4	Setup instructions	“Can I run this?” must be answerable
5	Reproducible environment	<code>.python-version</code> , pinned deps, <code>Makefile</code>
6	Working demo	One command to <i>see it work</i>
7	Automated tests	Green tests = you can change it safely
8	CI pipeline	Quality is enforced, not aspirational
9	Security scanning	gitleaks + pre-commit; no leaked secrets
10	Real benchmarks	Numbers, not adjectives
11	Evaluation report	You <i>measured</i> quality
12	Screenshots / charts	Skimmable proof
13	Known limitations	Honesty reads as senior
14	Cost analysis	You think about unit economics
15	Deployment instructions	You can ship, not just prototype
16	Demo script	You can present under time pressure
17	Interview talking points	Chapter 12

The “real numbers” bar. Items 10–11 are what separate a portfolio from a tutorial. A reviewer’s first probe is “is this real?” A repo that answers with `faithfulness 0.87`, `89.6 tok/s`, `p50 2.3 s`, `$0.00025/request` has already won the argument.

11.2 Turning “implemented” into “ready” — the concrete moves

For each repository, the specific actions that flip the last checkboxes:

- `prod-rag` — architecture diagram ☒, results chart ☒, eval + observability numbers ☒, CI gate wired ☒. *To finish:* set the `OPENAI_API_KEY` CI secret so the gate *enforces* (not skips); run the Langfuse stack

once for trace screenshots.

- **local-slm-lab** — benchmark + temperature tables , chart , structured- output before/after . *To finish:* grow the prompt set to 30–50 and add a quality rubric + quantization sweep (optional depth).
- **realtime-voice** — latency budget , chart , resilience + replay , Mermaid pipeline . *To finish:* add live provider keys for one real end-to-end latency capture.
- **llm-finetuning** — full pipeline + one-command `run_all.sh` , metrics code , diagram . *To finish:* run on a GPU box to fill the before/after table (the book's single **REQUIRES YOU** result set).

11.3 The portfolio scorecard

Status legend:  done ·  done, needs a manual/resource step to *activate* ·  awaiting your resources.

Capability	prod-rag (P1+P3)	local-slm-lab (P2)	llm-finetuning (P4)	realtime-voice (P5)
Clean structure + README				
Architecture diagram				
Reproducible env (<code>.python-version</code> , <code>Makefile</code>)				
Working demo (one command)			 (GPU)	
Automated tests				
CI pipeline				
Security scanning (gitleaks + pre-commit)				
Real benchmarks / eval	 0.87 / 0.85 / 1.00	 89.6 tok/s, temp study	 awaiting GPU	 ~202 ms (mock-stage baseline)
Observability	 p50 2.3s · \$0.00025 · 100% cite	—	—	 latency tracker
Results chart			—	
Deployment	 Docker + GHCR + ACA one-liner	container/edge	GPU job	container (WebSocket)
CI eval gate <i>enforced</i>	 set <code>OPENAI_API_KEY</code> secret	n/a	n/a	n/a
Live traces / live latency	 run Langfuse stack	n/a	n/a	 add provider keys

Reading the scorecard honestly

- **Three of four projects carry real, verified numbers today** (`prod-rag` , `local-slm-lab` , and `realtime-voice` 's instrumentation baseline).

- `llm-finetuning` **is a single GPU run from complete** — implemented, tested, one-command, and honest about the missing table rather than filling it with fiction.
- The **●** items are *activation* steps (a CI secret, a Docker stack, provider keys) — not missing engineering.

11.4 The hub: tie it together

The `ai-portfolio` hub repository presents all four projects on one page with a “results at a glance” scoreboard and cross-links. A reviewer should be able to land on the hub, read one table, and click into whichever project matches the role. Pin the four project repos on your GitHub profile and give each a description and topics (`rag` , `llm` , `fine-tuning` , `observability` , `real-time`).

REQUIRES YOU — final polish. Pin the repos, set descriptions/topics, and (optionally) record a ~90-second Loom walking one `prod-rag` query end to end (retrieve → rerank → cite → trace). A short video linked from each README is the single highest-leverage addition to a portfolio.

With the work polished and scored, the last step is presenting it — Chapter 12.

Chapter 12 — Hiring-Manager Presentation Guide

You built the systems; this chapter is how to *talk* about them. For each project you get a 30-second hook, a 2-minute technical summary, and the specific answers to the questions interviewers actually ask. The golden rule: **lead with a measured number, then explain the decision behind it.**

12.1 prod-rag (Projects 1 + 3)

- **30 seconds.** “A production RAG system over real Azure docs: hybrid retrieval, cross-encoder reranking, and enforced citations that make it *refuse* rather than hallucinate. It’s fully observable and CI-gated — faithfulness 0.87 with a build that fails if quality regresses.”
- **2 minutes.** “Ingestion chunks docs at 500–800 tokens with overlap and builds a Chroma vector index plus a BM25 index. At query time I fuse both with Reciprocal Rank Fusion, rerank the shortlist with a cross-encoder, then generate with `gpt-4o-mini` under a grounding prompt and enforce citations — if the answer isn’t supported, it abstains. Every request emits a Langfuse trace and a metrics record: I measured p50 2.3 s, p90 3.7 s, \$0.00025 per request, 100% citation coverage. Quality is a golden-set Ragas eval wired into CI as a gate at 0.80 / 0.78 / 0.80, so a PR that lowers faithfulness can’t merge.”
- **Main challenge:** making quality *non-regressable* — turning “seems good” into a threshold a build enforces.
- **Key decision:** citation enforcement + abstention (trust over coverage).
- **Main tradeoff:** hybrid + rerank add latency for accuracy; the p90 tail is generation, not retrieval, so it was worth it.
- **A failure & fix:** the eval initially returned `nan` for answer relevancy because I wasn’t passing an explicit judge LLM/embeddings to Ragas; I also found `nan` was slipping through the gate as a pass. I fixed the wrapper and made `nan` count as a *failure* — “no signal” must never read as green.
- **Most meaningful metric:** faithfulness — it directly measures hallucination.
- **In production I’d change:** grow the golden set and add Precision@K labels; sample live faithfulness, not just offline.
- **What I did / what needs resources:** all code, eval, observability, and CI are mine; enforcing the CI gate needs the `OPENAI_API_KEY` secret; live traces need the Langfuse stack running.

12.2 local-slm-lab (Project 2)

- **30 seconds.** “A fully offline model lab: I benchmarked three local models on identical hardware and prompts — Llama 3.2 hits 89.6 tokens/sec — and proved that plain prompting gives 0/5 valid JSON while a Pydantic+Instructor schema fixes it.”
- **2 minutes.** “The harness separates warm throughput from cold start via a warm-up pass, because the real production decision is whether to keep a model resident. Llama 3.2 (3B) decodes fastest at 89.6 tok/s; Mistral 7B has the lowest warm TTFT at 126 ms but an 8.4-second cold start. A temperature study shows temp 0 is fully deterministic (1 distinct output / 5 runs) and temp 0.7 diverges (4–5 / 5).

Crucially, even at temp 0 the model wrapped JSON in prose — 0/5 valid — so I enforce a schema with Instructor and retry-on-invalid.”

- **Key decision:** measure warm vs. cold separately.
- **Main tradeoff:** smaller model = higher throughput; larger model = better quality but an 8 s load penalty.
- **A failure & fix:** first JSON attempts were unparseable; the fix was to stop trusting the prompt and enforce a validated schema.
- **Most meaningful metric:** tokens/sec (throughput per GB drives serving cost).
- **What needs resources:** nothing — it’s fully offline.

12.3 llm-finetuning (Project 4)

- **30 seconds.** “A QLoRA + DPO pipeline that specialises a 4B model for structured JSON extraction, with an objective before/after scorecard and a one-command GPU run.”
- **2 minutes.** “I picked a task with crisp metrics — JSON validity, exact-match, refusal correctness — because fine-tuning only makes sense when you can score it. SFT with LoRA teaches the format; DPO on preference pairs, stacked on the SFT adapter, aligns the softer behaviours like clean refusals. QLoRA lets a 4B base train in ~16 GB and produces a few-MB adapter I can version and swap. The whole thing runs with `bash scripts/run_all.sh`.”
- **Key decision:** fine-tune for *skill/format*, retrieve for *facts*.
- **Main tradeoff:** QLoRA vs. full fine-tuning — comparable quality for a fraction of VRAM and storage.
- **What needs resources:** a CUDA GPU to produce the before/after numbers — I’m explicit that those are not yet measured rather than inventing them.
- **Interview honesty win:** “If DPO doesn’t beat SFT, that’s a real finding — it means the preference pairs weren’t discriminative, and the fix is better data, not more epochs.”

12.4 realtime-voice (Project 5)

- **30 seconds.** “A streaming voice assistant built around a latency budget — ~202 ms mouth-to-ear across ASR, LLM, and TTS — with timeouts, fallbacks, and a replay mode so failures are debuggable.”
- **2 minutes.** “It’s ASR → LLM → TTS over a pipeline where each stage is timed and every external call has three planned outcomes: success, slow (timeout → fallback), and failure (circuit-break → degrade). The instrumentation measures per-stage p50/p90 and sums them to the user-perceived delay. The same code runs fully mocked (no keys) or with real providers per key set. A replay mode reproduces recorded sessions deterministically, and failure injection verifies the user always gets a timely, useful response — never dead air.”
- **Key decision:** measure *per stage*, not just end-to-end.
- **Main tradeoff:** graceful degradation adds complexity but is the difference between “feels real” and “hangs.”
- **Most meaningful metric:** end-to-end p50 (the felt delay).
- **What needs resources:** provider keys for a live latency capture; the instrumentation and resilience are real and testable offline.

12.5 The question bank

Strong, concise answers to the questions these projects invite.

Why use hybrid retrieval? BM25 matches exact terms; vectors match meaning. Real questions mix both, so fusing them with RRF beats either alone — and RRF is rank-based, so it needs no score calibration.

Why a cross-encoder reranker? A bi-encoder embeds query and passage separately for speed; a cross-encoder reads them together for accuracy. You can't afford the cross-encoder over the whole corpus, so you use it to reorder the shortlist — best of both.

How did you measure faithfulness? Ragas with `gpt-4o-mini` as judge over an 18-pair golden set; it checks whether each claim is supported by the retrieved context. I gated it in CI at 0.80 and made `nan` a failure.

Why a local model instead of an API? Privacy/compliance, latency, cost at scale, edge/offline. I quantified the cost: 89.6 tok/s and a few GB of memory for a 3B model — enough for a bounded task with zero data egress.

How did you benchmark tokens/sec? Fixed hardware, prompt set, and generation settings; a warm-up pass to isolate cold start; throughput from Ollama's `eval_count / eval_duration`. Warm and cold reported separately.

Why QLoRA instead of full fine-tuning? ~16 GB vs. tens of GB VRAM, a few-MB swappable adapter vs. a full checkpoint, comparable quality for a single-task skill.

What did DPO improve? It aligns preferences SFT can't — cleaner refusals, fewer hallucinated fields — stacked on the SFT adapter. And if it doesn't help, that's a data-quality signal.

How did you prevent evaluation leakage? Golden answers derived from the docs, never shown to the model; disjoint train/val/test with cross-split dedup; thresholds tuned on validation, reported on held-out test.

How did you calculate latency percentiles? Record every request's per-stage and end-to-end latency to a log, then compute p50/p90 (p95/p99 with enough volume). I report percentiles, never averages, because the tail is the user who churns.

How does the system degrade when a provider fails? Every call has a timeout → fallback and a circuit breaker → degraded mode. TTS fails → text fallback; LLM times out → spoken acknowledgement; WebSocket drops → session recovers. Never dead air.

How would you scale it? Stateless API behind a load balancer, scale-to-zero on a serverless container platform, immutable tagged images, cache and cap model concurrency, and alert on p90/cost/failure.

How would you reduce cost? Tighter reranked context (fewer input tokens), a smaller or local model where quality allows, and caching. My measured unit cost is \$0.00025/request, so I can project spend precisely.

How would you secure the application? Secrets in a platform secret store never in git (gitleaks + pre-commit enforce this), least-privilege keys, dependency scanning, and input validation at the API boundary.

What would you monitor in production? p50/p90/p95/p99 latency, cost/request, failure rate, citation coverage, empty-retrieval and retry rates, and sampled faithfulness — with alerts on the tail and on quality drift.

Closing principle for every answer. State the number, name the decision, admit the tradeoff. “faithfulness 0.87 because I enforce citations and abstain; the cost is coverage on thin-evidence questions, which I accept for trust.” That cadence — measured, deliberate, honest — is what gets the offer.

Chapter 13 — Bibliography and Further Reading

Sources are grouped by theme. Where a primary paper exists it is cited alongside the official documentation an engineer actually uses day to day.

Foundations: transformers, tokens, embeddings

- Vaswani, A. et al. (2017). *Attention Is All You Need*. NeurIPS.
- Devlin, J. et al. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. NAACL.
- Reimers, N. & Gurevych, I. (2019). *Sentence-BERT*. EMNLP.
- Liu, N. et al. (2023). *Lost in the Middle: How Language Models Use Long Contexts*.
- OpenAI Tokenizer documentation; Hugging Face LLM Course.
- Raschka, S. *Build a Large Language Model (From Scratch)*. (stylistic model for this book.)

Retrieval-Augmented Generation

- Lewis, P. et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS.
- Robertson, S. & Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond*.
- Cormack, G. et al. (2009). *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods*. SIGIR.
- Nogueira, R. & Cho, K. (2019). *Passage Re-ranking with BERT*.
- ChromaDB, sentence-transformers, and Cohere Rerank documentation.

Evaluation and observability

- Es, S. et al. (2023). *RAGAS: Automated Evaluation of Retrieval Augmented Generation*. Ragas documentation.
- Langfuse, LangSmith, and Braintrust documentation.
- OpenTelemetry specification (traces, spans, context propagation).
- Beyer, B. et al. (2016). *Site Reliability Engineering* (Google) — the Four Golden Signals and percentile-based monitoring.

Local models and structured output

- Ollama documentation and model library.
- Model cards: Llama 3.2 (Meta), Phi-4 (Microsoft), Mistral 7B (Mistral AI).
- Instructor (python.useinstructor.com) and Pydantic documentation.
- `llama.cpp` / GGUF quantization notes (K-quants).

Fine-tuning: LoRA, QLoRA, DPO

- Hu, E. et al. (2021). *LoRA: Low-Rank Adaptation of Large Language Models*.
- Dettmers, T. et al. (2023). *QLoRA: Efficient Finetuning of Quantized LLMs*.
- Rafailov, R. et al. (2023). *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*.
- Ouyang, L. et al. (2022). *Training Language Models to Follow Instructions with Human Feedback* (InstructGPT / RLHF).
- Hugging Face TRL and PEFT documentation; Axolotl documentation; Qwen3 model card; dataset cards for `ultrachat_200k` and `distilabel-intel-orca-dpo-pairs`.

Real-time systems

- Nygard, M. (2018). *Release It!* (2nd ed.) — circuit breakers, bulkheads, stability patterns.
- Fette, I. & Melnikov, A. (2011). *The WebSocket Protocol* (RFC 6455).
- Pipecat documentation; Deepgram, OpenAI, Anthropic, ElevenLabs, and Cartesia API documentation.

Engineering practice: Git, CI, deployment, security

- Chacon, S. & Straub, B. *Pro Git*.
- Conventional Commits; Semantic Versioning (semver.org); Keep a Changelog.
- Gitleaks and pre-commit documentation; GitHub Actions, branch protection, and Dependabot documentation; `pip-audit`.
- Docker and Docker Compose documentation; GitHub Container Registry.
- Azure Container Apps, AWS App Runner/Fargate, Google Cloud Run, and Kubernetes documentation.
- Wiggins, A. *The Twelve-Factor App*.
- OWASP Top 10 and OWASP testing guidance.

Colophon

This textbook was written to accompany four public repositories under `github.com/mimuruth` (`prod-rag`, `local-slm-lab`, `llm-finetuning`, `realtime-voice`) and the `ai-portfolio` hub. Every number tagged **MEASURED** was produced on the hardware in Chapter 2; every **REQUIRES YOU** item names the exact manual, credentialed, or GPU step needed to complete it. The PDF is generated from Markdown by a reproducible Node + Puppeteer pipeline in `ai-portfolio/book/`; diagrams are rendered with Mermaid and charts with Matplotlib from the projects' own result files. Nothing here is a fabricated benchmark — that discipline is the whole point.